
Software Requirements Specification

for

AI-BASED EMPTY-RETURN TRUCK SHARING AND FREIGHT OPTIMIZATION SYSTEM

Version 1.0 approved

Prepared by Rohithraghav S

Coimbatore Institute of Technology

11.06.25

TABLE OF CONTENTS

Table of Contents	ii
Revision History	ii
1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	5
1.4 Project Scope	6
1.5 References	6
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Features	8
2.3 User Classes and Characteristics	10
2.4 Operating Environment	15
2.5 Design and Implementation Constraints	15
2.6 User Documentation	16
2.7 Assumptions and Dependencies	16
3. System Features	17
3.1 User Registration and Authentication Module	17
3.2 Return Route Registration Module	18
3.3 Freight Search and AI Route Matching Module	19
3.4 Booking and Capacity Allocation Module	20
4. External Interface Requirements	21
4.1 User Interfaces	21
4.2 Hardware Interfaces	22
4.3 Software Interfaces	23
4.4 Communication Interfaces	24
5. Other Non-functional Requirements	24
5.1 Performance Requirements	24
5.2 Safety Requirements	25
5.3 Security Requirements	25
5.4 Software Quality Attributes	26

5.5 Business Rules	27
6. Other Requirements	27
6.1 Database Requirements	27
6.2 Internationalization Requirements	28
6.3 Legal and Regulatory Requirements	28
6.4 Reuse Objectives	29
6.5 Future Enhancements	29
Appendix A: Glossary	29
Appendix B: Analysis Models	31
Appendix C: Issues List	39

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define and describe the requirements for the AI-Based Empty-Return Truck Sharing and Freight Optimization System. The system is designed to improve logistics efficiency by reducing the number of empty-return truck trips and maximizing the utilization of available transportation resources.

In the transportation and logistics industry, a significant number of commercial trucks return empty after delivering goods to their destinations. These empty-return trips result in wasted fuel, increased transportation costs, reduced profitability for logistics companies, and higher environmental pollution due to unnecessary carbon emissions. Simultaneously, many small-scale businesses and merchants struggle to find affordable freight transportation for low-volume shipments.

The proposed system provides a centralized web-based platform where truck providers can register their return routes and available cargo capacities. Shippers can search for transportation opportunities along these routes and reserve cargo space at optimized costs. The system incorporates an intelligent route-matching mechanism that identifies suitable transportation opportunities based on predefined route corridors and available capacity.

This document serves as a formal agreement between stakeholders, software developers, testers, project supervisors, and evaluators. It specifies all functional requirements, non-functional requirements, constraints, assumptions, interfaces, and system features necessary for successful software development and deployment.

1.2 Document Conventions

This document follows the structure and recommendations provided in the IEEE Standard 830-1998 Software Requirements Specification format. The requirements presented throughout this document are categorized and numbered systematically to ensure clarity, traceability, and ease of maintenance.

The following conventions are used throughout the document:

- **Requirement Identifiers**
- Functional Requirements are represented using the notation **FR-x**.
- Non-Functional Requirements are represented using the notation **NFR-x**.
- Business Rules are represented using the notation **BR-x**.
- **User Roles**

The system recognizes the following primary user roles:

- Truck Provider
- Shipper
- Administrator

- AI Matching Engine (System Component)

Terminology

Important technical terms such as Route Corridor, Capacity Allocation, Freight Booking, and Route Matching are defined in Appendix A (Glossary).

Priority Classification

Requirements are categorized according to their importance:

- **High Priority:** Essential for core system functionality.
- **Medium Priority:** Important but not critical for initial deployment.
- **Low Priority:** Optional enhancements that may be included in future releases.

All requirement statements use the word "shall" to indicate mandatory functionality.

1.3 Intended Audience and Reading Suggestions

This Software Requirements Specification document is intended for various stakeholders who participate in the development, testing, deployment, and use of the system.

Software Developers

Software developers will use this document as the primary reference for understanding system functionality, business rules, database structures, and implementation requirements. It provides detailed specifications necessary for system design and coding activities.

Test Engineers

Test engineers will use this document to develop test cases, verify system functionality, validate software requirements, and perform quality assurance activities. Functional and non-functional requirements defined in this document will serve as acceptance criteria during testing.

Project Supervisors and Evaluators

Faculty members, project guides, and external evaluators will use this document to understand the project scope, objectives, system architecture, and software engineering practices adopted during development.

Logistics Companies and Truck Providers

Truck providers can understand how the system supports route registration, capacity management, and freight booking operations.

Shippers and Business Users

Shippers can understand the booking process, route search functionality, and freight management capabilities available through the system.

System Administrators

Administrators responsible for maintaining the platform can use this document to understand monitoring capabilities, user management functions, and operational requirements.

Readers are encouraged to begin with the Introduction and Overall Description sections before reviewing detailed functional and non-functional requirements.

1.4 Project Scope

The AI-Based Empty-Return Truck Sharing and Freight Optimization System is a web-based logistics platform designed to improve transportation efficiency by reducing empty truck return trips and facilitating shared freight transportation.

The system acts as an intermediary platform connecting truck providers who have unused cargo capacity during return journeys with shippers seeking affordable freight transportation services. Through intelligent route matching and capacity management, the platform optimizes vehicle utilization and reduces transportation waste.

The primary objectives of the system are:

- Reduce empty-return truck trips.
- Increase utilization of available transportation resources.
- Minimize fuel consumption and transportation costs.
- Reduce environmental impact through lower carbon emissions.
- Provide affordable freight services for small and medium-sized businesses.
- Improve operational efficiency within logistics networks.
- Support sustainable transportation practices.

The system employs a route corridor matching mechanism that compares source and destination locations against predefined truck return routes. Instead of relying on complex GPS-based optimization systems, the application uses route arrays and corridor analysis to simulate intelligent matching while maintaining implementation simplicity suitable for academic projects.

The software can be deployed by logistics companies, freight operators, supply chain organizations, transportation startups, and third-party freight brokers.

1.5 References

The following references were used during the preparation of this Software Requirements Specification:

1. IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
2. Transportation and Logistics Management Principles.
3. Freight Optimization and Supply Chain Management Research Publications.
4. PostgreSQL Official Documentation.
5. FastAPI Official Documentation.
6. Software Engineering: A Practitioner's Approach by Roger S. Pressman.
7. Unified Modeling Language (UML) Specifications.
8. Transportation Management System (TMS) Best Practices.
9. Software Project Management Guidelines.
10. Web Application Security Best Practices.

2. Overall Description

2.1 Product Perspective

The AI-Based Empty-Return Truck Sharing and Freight Optimization System is an independent web-based application developed specifically to address inefficiencies associated with empty-return freight transportation.

The system serves as a centralized digital marketplace that connects truck providers and shippers through an automated freight-sharing mechanism. It enables transportation providers to advertise available cargo capacity on return journeys while allowing businesses to identify and reserve suitable freight space.

The application follows a three-tier architecture:

Presentation Layer

The presentation layer consists of web-based interfaces that enable interaction between users and the system. This layer provides separate dashboards for truck providers, shippers, and administrators.

Business Logic Layer

The business logic layer handles all core system operations including:

- User authentication
- Route registration
- Freight searching
- AI route matching
- Capacity management
- Booking processing
- Administrative monitoring

Database Layer

The database layer stores all persistent information including:

- User accounts
- Truck routes
- Booking records
- Capacity details
- Transaction history
- System logs

The proposed system can be integrated with future enhancements such as:

- GPS tracking systems
- Real-time vehicle monitoring
- Online payment gateways
- Mobile applications
- Machine learning-based route optimization
- Demand forecasting systems

2.2 Product Features

The major functions of the AI-Based Empty-Return Truck Sharing and Freight Optimization System include:

User Registration and Authentication

The system provides secure registration and login facilities for truck providers, shippers, and administrators. User credentials are validated and securely stored within the database.

Route Registration

Truck providers can register return routes by specifying:

- Source city
- Intermediate transit hubs
- Destination city
- Vehicle capacity
- Freight pricing information

Freight Search

Shippers can search available freight opportunities by entering:

- Pickup location
- Delivery location
- Cargo weight
- Cargo volume

AI-Based Route Matching

The route-matching engine automatically compares shipper requirements against available truck routes. It identifies matching route corridors and verifies capacity availability before presenting results.

Booking Management

Shippers can reserve cargo space on available trucks. The system generates unique booking identifiers and records transaction details.

Capacity Allocation

The system automatically updates truck capacity after successful bookings. Available cargo space is recalculated in real time to prevent overbooking.

Freight Tracking

Users can view booking details, route information, and booking status updates through personalized dashboards.

Administrative Monitoring

Administrators can monitor:

- Registered users
- Active routes
- Freight bookings
- System performance

- Operational reports

Reporting and Analytics

The system maintains operational records and generates reports related to:

- Route utilization
- Booking trends
- Vehicle occupancy rates
- Revenue statistics

Driver Management

The system allows drivers to:

- View assigned return routes.
- Accept assigned trips.
- Update trip progress.
- Confirm cargo pickup.
- Confirm cargo delivery.
- Report route completion.

2.3 User Classes and Characteristics

Truck Provider

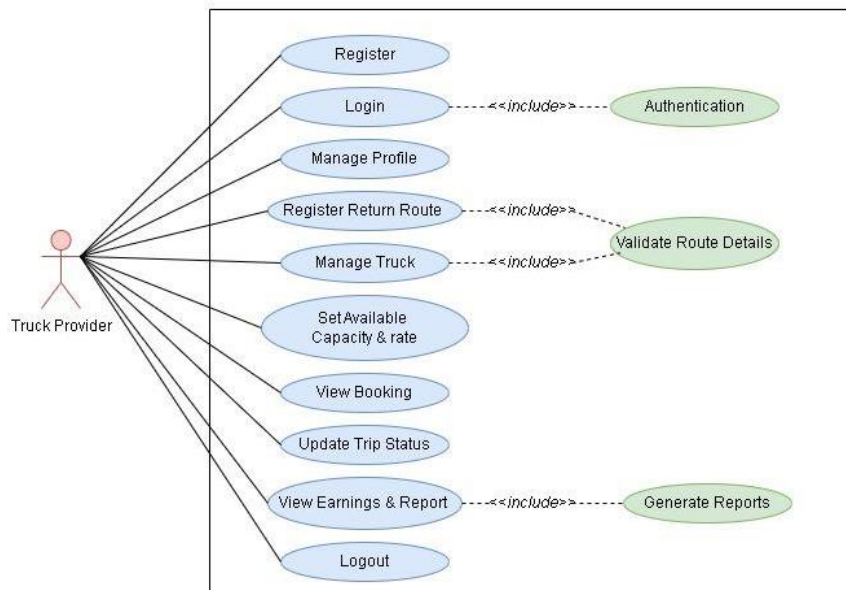
Truck providers are logistics companies, transportation firms, fleet operators, or independent truck owners.

Characteristics:

- Moderate technical expertise.
- Responsible for route registration.
- Manage available truck capacity.
- View active bookings.
- Update transportation information.

Priority Level: High

Use Case Diagram:



Use Case Description: Search Available Trucks

Primary Actor: Shipper

Secondary Actor: System

Pre condition: Shipper is registered and logged in.

Post condition: Optimized matching truck routes are listed for selection.

Trigger: Shipper selects Search Available Trucks.

Main Success Scenario:

1. Shipper logs into the system.
2. Selects Search Available Trucks.
3. System prompts for search criteria (Origin, Destination, Date, Cargo Type).
4. Shipper inputs the criteria and submits.
5. System triggers "Search & Match Routes" using the AI matching engine.
6. System displays prioritized matching options.

7. Confirmation of matching results is displayed.

Exception Scenario:

- Missing or invalid location inputs.
- No matching truck routes found for criteria

Driver

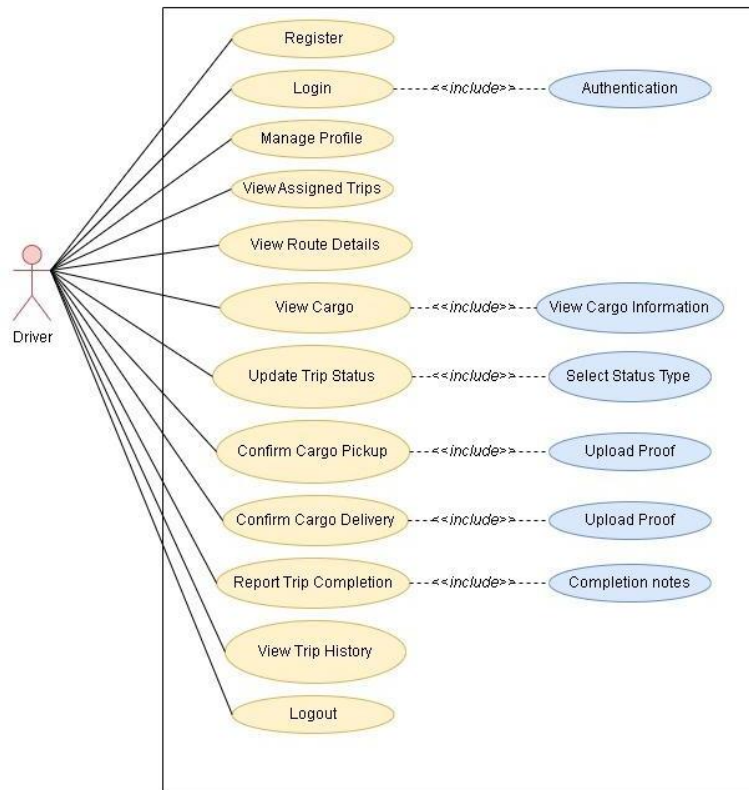
Drivers are responsible for operating the registered trucks and updating trip-related information during transportation. They access the system to view assigned trips, update delivery status, confirm cargo loading and unloading, and communicate with truck providers and administrators.

Characteristics:

- Basic to moderate technical knowledge.
- Uses a smartphone or web application.
- Views assigned routes.
- Updates trip status.
- Confirms cargo loading and delivery.
- Reports delivery completion.

Priority Level: High

Use Case Diagram:



Use case Description: Update Trip Status

Primary Actor: Driver

Secondary Actor: System

Pre condition: Driver has started an assigned transit route.

Post condition: Live shipment status flag is modified on the server.

Trigger: Driver selects Update Trip Status.

Main Success Scenario:

1. Driver opens the trip management window.
2. Selects Update Trip Status.
3. System triggers the <<include>> use case: Select Status Type.
4. System presents specific transit state choices (e.g., En Route, Delayed, Resting).
5. Driver selects the applicable state option.
6. System updates the live tracking index.
7. Confirmation notification is displayed.

Exception Scenario:

- Data validation conflict from trying to select an identical or repeated status state.

Shipper

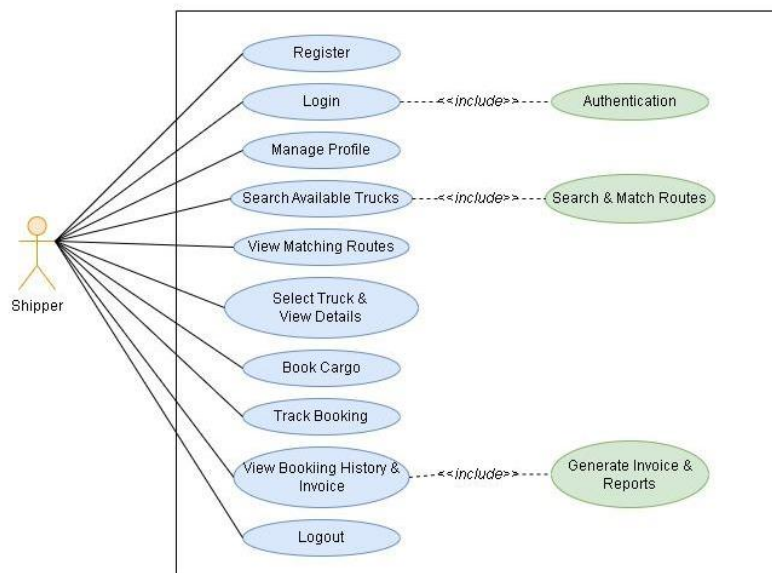
Shippers are businesses or individuals requiring freight transportation services.

Characteristics:

- Basic computer literacy.
- Search available freight routes.
- Reserve cargo space.
- Track booking status.
- View booking history.

Priority Level: High

Use Case Diagram:



Use Case Description: Set Available Capacity & Rate

Primary Actor: Truck Provider

Secondary Actor: System

Pre condition: Truck Provider has an active registered truck profile and is logged into the system.

Post condition: Space constraints and quote pricing are configured for the asset.

Trigger: Truck Provider selects Set Available Capacity & Rate.

Main Success Scenario:

1. Truck Provider logs into the system.
2. Selects Set Available Capacity & Rate.
3. System prompts for volume/weight boundaries and price per mile.
4. Truck Provider enters values.
5. System validates numeric constraints.
6. System updates search parameters across matching indices.
7. Confirmation message is displayed.

Exception Scenario:

- Negative or zero pricing entries submitted.
- Dimensions exceed original truck specifications.

Administrator

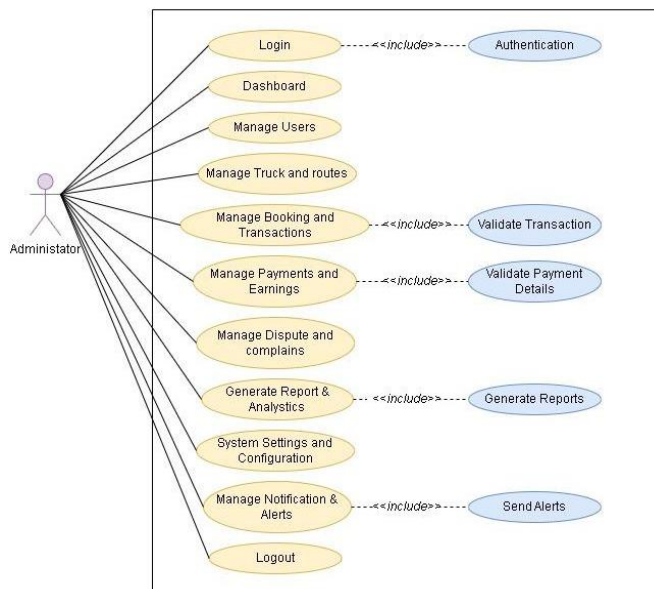
Administrators are responsible for system management and monitoring.

Characteristics:

- Advanced technical knowledge.
- Monitor platform activity.
- Manage users.
- Generate reports.
- Resolve disputes.

Priority Level: Medium

Use Case Diagram:



Use Case: Manage Payments and Earnings

Primary Actor: Administrator

Secondary Actor: System

Pre condition: Administrator portal login verified and active ledger access granted.

Post condition: Financial disbursements cleared and balances settled.

Trigger: Administrator selects Manage Payments and Earnings.

Main Success Scenario:

1. Administrator accesses ledger dashboard panels.
2. Selects Manage Payments and Earnings.
3. System prompts options to process shipper collections or driver payouts.
4. Administrator executes batch release actions.
5. System triggers "Validate Payment Details" secure processing sequence.
6. System processes third-party payment gateway transaction APIs.
7. Confirmation message is displayed.

Exception Scenario:

- Payment details validation error (invalid bank routing numbers).
- Gateway timeout errors during external banking handshake procedures.

2.4 Operating Environment

The system shall operate within the following environment:

Frontend Technologies

- HTML5
- CSS3
- JavaScript

Backend Technologies

- FastAPI Framework
- Python Programming Language
- **Database System**
- PostgreSQL Version 14 or later

Operating Systems

- Ubuntu Linux
- Windows 10
- Windows 11

Browser Support

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Development Tools

- Visual Studio Code
- pgAdmin
- GitHub
- Postman

2.5 Design and Implementation Constraints

The following constraints apply to the system:

- The backend shall be implemented using FastAPI.
- PostgreSQL shall be used as the database management system.
- Route matching shall be implemented using predefined route corridor arrays.
- Mobile-responsive design shall be supported.
- HTTPS communication shall be enforced.
- Passwords shall be stored using encryption or hashing techniques.
- The system shall support concurrent users.
- The application shall follow modular design principles.
- Database normalization shall be applied.

2.6 User Documentation

The following documentation shall be provided with the software:

- **User Manual**
Provides instructions for registration, route searching, booking, and account management.
- **Administrator Guide**
Provides guidance for monitoring users, routes, and bookings.
- **Installation Guide**
Provides deployment instructions for server administrators.
- **Technical Documentation**
Provides architecture diagrams, database schemas, and API documentation.
- **Help and FAQ Section**
Provides answers to common user questions.

2.7 Assumptions and Dependencies

Assumptions

- Users have access to reliable internet connectivity.
- Truck providers provide accurate route information.
- Freight details entered by shippers are accurate.
- Users possess basic web browsing knowledge.

Dependencies

- PostgreSQL Database Server.
- FastAPI Runtime Environment.
- SMTP Email Notification Service.
- Web Hosting Infrastructure.
- Modern Web Browsers.

The success of the project depends upon the availability and reliability of these external components and services.

3. System Features

3.1 User Registration and Authentication Module

3.1.1 Description and Priority

This feature enables Truck Providers, Shippers, and Administrators to create secure user accounts and access the system based on their assigned roles.

Priority: High

Benefit: 9

Penalty: 8

Cost: 4

Risk: 3

This module ensures that only authorized users can access protected resources within the system.

3.1.2 Stimulus/Response Sequence

1. User visits the registration page.
2. User enters personal and login details.
3. System validates all mandatory fields.
4. User account is created successfully.
5. User logs into the application.
6. System authenticates credentials.
7. User is redirected to the appropriate dashboard.

3.1.3 Functional Requirements

FR-1: The system shall allow users to register as either Truck Providers or Shippers.

FR-2: The system shall validate all registration details before account creation.

FR-3: The system shall prevent duplicate email registrations.

FR-4: The system shall encrypt user passwords before storing them in the database.

FR-5: The system shall authenticate registered users using secure login credentials.

FR-6: The system shall redirect authenticated users to role-specific dashboards.

3.2 Return Route Registration Module

3.2.1 Description and Priority

This feature enables Truck Providers to register empty-return routes and specify available cargo capacity.

Priority: High

Benefit: 9

Penalty: 9

Cost: 5

Risk: 4

3.2.2 Stimulus/Response Sequence

1. Truck Provider logs into the system.
2. Opens the "Register Return Route" page.
3. Enters source city, intermediate hubs, destination city, return date, available capacity, and freight rate.
4. Clicks the Submit button.
5. System validates the entered information.
6. Route information is stored successfully.
7. Route becomes available for freight matching.

3.2.3 Functional Requirements

FR-7: The system shall allow Truck Providers to register return routes.

FR-8: The system shall store route corridor information.

FR-9: The system shall maintain available cargo capacity.

FR-10: The system shall allow providers to edit or delete registered routes before departure.

FR-11: The system shall display all active routes in the provider dashboard.

3.3 Freight Search and AI Route Matching Module

3.3.1 Description and Priority

This feature enables Shippers to search available freight opportunities. The AI matching engine compares requested routes with registered return corridors and identifies suitable transportation options.

Priority: High

Benefit: 9

Penalty: 9

Cost: 5

Risk: 5

3.3.2 Stimulus/Response Sequence

1. Shipper logs into the system.
2. Opens the Freight Search page.
3. Enters pickup city, destination city, and cargo weight.
4. Clicks the Search button.
5. AI Matching Engine compares the requested route with available truck routes.
6. Matching routes are displayed.
7. User selects the preferred truck.

3.3.3 Functional Requirements

FR-12: The system shall accept freight search requests.

FR-13: The system shall compare requested routes with registered return corridors.

FR-14: The system shall verify cargo capacity before displaying matching results.

FR-15: The system shall display only available trucks satisfying both route and capacity requirements.

FR-16: The system shall notify the user when no matching route is available.

3.4 Booking and Capacity Allocation Module

3.4.1 Description and Priority

This module enables shippers to reserve available cargo space on matched trucks and automatically updates vehicle capacity after successful booking.

Priority: High

Benefit: 9

Penalty: 8

Cost: 5

Risk: 3

3.4.2 Stimulus/Response Sequence

1. User selects a matched truck.
2. Booking details are displayed.
3. User confirms the reservation.
4. System verifies remaining capacity.
5. Booking record is created.
6. Unique Booking ID is generated.
7. Remaining truck capacity is updated automatically.
8. Confirmation message is displayed.

3.4.3 Functional Requirements

FR-17: The system shall generate a unique Booking ID for every successful reservation.

FR-18: The system shall deduct booked cargo weight from the available truck capacity.

FR-19: The system shall maintain booking history for both Truck Providers and Shippers.

FR-20: The system shall prevent bookings when available capacity is insufficient.

FR-21: The system shall display booking confirmation immediately after successful reservation.

4.5 Driver Management Module

4.5.1 Description and Priority

This feature enables registered drivers to manage assigned return trips and update transportation status throughout the delivery process.

Priority: High

Benefit: 8

Penalty: 8

Cost: 4

Risk: 3

4.5.2 Stimulus/Response Sequences

1. Driver logs into the system.
2. System displays assigned return trips.
3. Driver selects a trip.
4. System displays truck, cargo, and route details.
5. Driver updates trip status.
6. Driver confirms cargo loading.
7. Driver confirms cargo delivery.
8. System updates booking records.
9. Administrator and shipper receive status notifications.

4.5.3 Functional Requirements

FR-22: The system shall allow drivers to log in securely.

FR-23: The system shall display all assigned trips.

FR-24: The system shall allow drivers to update trip status.

FR-25: The system shall allow drivers to confirm cargo pickup.

FR-26: The system shall allow drivers to confirm cargo delivery.

FR-27: The system shall notify shippers whenever the trip status changes.

FR-28: The system shall record the completion time of each trip.

4. External Interface Requirements

4.1 User Interfaces

The AI-Based Empty-Return Truck Sharing and Freight Optimization System will provide a modern, responsive, and user-friendly web interface that can be accessed through standard web browsers on desktop computers, laptops, tablets, and smartphones. The interface is designed to simplify freight booking and route management for all categories of users.

The application provides separate dashboards for Truck Providers, Shippers, and Administrators. Each dashboard displays only the functions relevant to the user's role. The interface follows a consistent design with a navigation bar, sidebar menu, and dashboard cards for easy navigation.

The system includes the following user interfaces:

- **Login Page:** Allows registered users to securely log in using their email address and password.
- **Registration Page:** Enables new Truck Providers and Shippers to create accounts by entering their personal and business details.
- **Truck Provider Dashboard:** Allows providers to register return routes, specify available cargo capacity, update trip details, and view existing bookings.
- **Shipper Dashboard:** Enables shippers to search available freight routes, compare matching trucks, reserve cargo space, and view booking history.
- **Booking Page:** Displays booking details, estimated transportation cost, remaining truck capacity, and booking confirmation.
- **Administrator Dashboard:** Allows administrators to monitor registered users, manage routes, resolve disputes, and generate reports.
- **Driver Dashboard:** The Driver Dashboard allows drivers to: View assigned trips, View route details, View cargo information, Update trip status, Confirm cargo loading, Confirm cargo delivery, Report completed journeys.

The interface shall maintain consistency by providing common navigation menus, standard action buttons (Save, Update, Delete, Cancel, Search, Book), and informative validation messages. Error messages shall be displayed beside the corresponding input fields, and confirmation dialogs shall appear before performing critical operations such as deleting routes or cancelling bookings.

The application shall also support keyboard navigation, responsive layouts, and accessibility guidelines to ensure ease of use for all users.

4.2 Hardware Interfaces

Since the proposed application is completely web-based, it does not require specialized hardware devices for operation. The system can be accessed through commonly available computing devices.

The supported hardware platforms include:

- Desktop Computers
- Laptop Computers
- Smartphones
- Tablets

Minimum hardware requirements include:

- Intel Core i3 Processor or equivalent
- 4 GB RAM
- 20 GB Free Disk Space
- Stable Internet Connection

The application is designed to support standard input devices including:

- Keyboard
- Mouse
- Touchscreen
- Touchpad

No external sensors or dedicated logistics hardware are required for the implementation of this project.

4.3 Software Interfaces

The software interfaces of the proposed system include:

Operating System

- Windows 10/11
- Ubuntu Linux
- macOS

Web Browser

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Backend Framework

FastAPI is used to implement RESTful APIs and business logic responsible for user authentication, route management, booking processing, and freight matching.

Database

PostgreSQL serves as the centralized relational database management system for storing user information, route details, bookings, and transaction history.

Development Tools

- Visual Studio Code
- pgAdmin
- GitHub
- Postman

The application communicates with the PostgreSQL database using SQL queries through the backend framework.

4.4 Communication Interfaces

Communication between various software components shall be established using standard Internet protocols.

The communication interfaces include:

- HTTP/HTTPS Protocol for secure communication between client and server.
- REST APIs for data exchange between frontend and backend.
- JSON format for transmitting data between application layers.
- SMTP protocol for sending email notifications regarding booking confirmations and account registration.

Security mechanisms include:

- SSL/TLS encryption
- Secure user authentication
- Session management
- Input validation
- Password hashing

Future versions may integrate communication with GPS services, online payment gateways, and SMS notification services.

5. Other Nonfunctional Requirements

Non-functional requirements define the quality attributes, performance expectations, reliability, security, and operational standards of the AI-Based Empty-Return Truck Sharing and Freight Optimization System. These requirements ensure that the system performs efficiently, securely, and reliably under different operating conditions.

5.1 Performance Requirements

The AI-Based Empty-Return Truck Sharing and Freight Optimization System shall provide efficient and reliable performance to support smooth logistics operations. The following performance requirements shall be satisfied:

- The system shall respond to all user requests within **2 seconds** under normal operating conditions.
- The AI route matching algorithm shall identify suitable truck routes and display search results within **3 seconds** after receiving the search request.
- The system shall support at least **100 concurrent users** without significant degradation in response time.
- Booking confirmation and capacity updates shall be completed within **2 seconds** after successful booking.
- The database shall support multiple simultaneous read and write operations while maintaining data consistency.
- The system shall maintain an availability of at least **99.5%** during normal operational periods.
- The application shall efficiently handle increasing numbers of users, truck routes, and freight bookings without affecting system performance.
- The route matching engine shall accurately process route corridor comparisons without excessive computational delay.

5.2 Safety Requirements

The system shall ensure safe operation and prevent accidental data loss or inconsistent system states.

- Automatic database backups shall be performed periodically to prevent data loss.
- The system shall validate all user inputs before processing any request.
- Critical operations such as deleting routes, cancelling bookings, or modifying capacity shall require user confirmation.
- The system shall prevent overbooking by verifying available truck capacity before confirming reservations.
- Database transactions shall support rollback mechanisms in case of unexpected failures.
- The system shall generate appropriate error messages without exposing internal implementation details.
- Unauthorized modifications to booking records shall be prevented.
- The system shall maintain audit logs for important administrative activities.

5.3 Security Requirements

Security is an essential aspect of the proposed system because it stores sensitive user information and booking data.

The system shall satisfy the following security requirements:

- User passwords shall be encrypted using secure hashing algorithms such as **bcrypt**.
- All communication between client and server shall occur through **HTTPS** using SSL/TLS encryption.

- Only authenticated users shall be allowed to access protected system resources.
- Role-Based Access Control (RBAC) shall restrict system functions based on user roles (Truck Provider, Shipper, Administrator).
- SQL Injection attacks shall be prevented through parameterized database queries.
- Cross-Site Scripting (XSS) vulnerabilities shall be prevented using input validation and output encoding.
- Cross-Site Request Forgery (CSRF) protection shall be implemented wherever applicable.
- User sessions shall automatically expire after a specified period of inactivity.
- Login attempts shall be monitored to prevent brute-force attacks.
- System activity logs shall be maintained for auditing and security monitoring purposes.

5.4 Software Quality Attributes

The proposed system shall satisfy the following software quality attributes:

Usability

The application shall provide a simple, intuitive, and user-friendly interface that enables users with basic computer knowledge to perform their tasks without extensive training. Navigation menus, buttons, forms, and messages shall be easy to understand.

Reliability

The system shall provide accurate route matching, consistent booking operations, and reliable database transactions. It shall recover gracefully from unexpected failures without causing data corruption.

Maintainability

The software shall follow modular programming principles so that future enhancements and bug fixes can be implemented with minimal changes to the existing codebase.

Scalability

The system architecture shall support future expansion, including additional users, vehicles, cities, payment gateways, GPS integration, and mobile applications.

Availability

The system shall remain operational throughout normal business hours with minimal downtime for maintenance activities.

Portability

The application shall operate on multiple operating systems including Windows and Linux and shall be accessible using modern web browsers.

Interoperability

The software shall support integration with external systems such as payment gateways, GPS tracking services, SMS gateways, and logistics management platforms.

Testability

The application shall support unit testing, integration testing, and system testing through modular software architecture and clearly defined APIs.

5.5 Business Rules

The following business rules govern the operation of the AI-Based Empty-Return Truck Sharing and Freight Optimization System:

- Only registered Truck Providers are allowed to publish return routes.
- Only authenticated Shippers are allowed to reserve freight space.
- Every registered truck shall have a unique Route ID.
- Every booking shall generate a unique Booking ID.
- Truck capacity shall never become negative after booking.
- Bookings shall only be confirmed when sufficient cargo capacity is available.
- A truck route marked as "Completed" shall no longer accept new bookings.
- Administrators shall have permission to monitor all routes and bookings.
- Only Administrators shall have permission to delete user accounts.
- Freight charges shall be calculated based on booked cargo weight and the provider's specified rate.
- Users shall be responsible for maintaining accurate profile and route information.
- Only registered drivers shall operate assigned trucks.
- A driver may be assigned to multiple trips but only one active trip at a time.
- Drivers shall update trip status during transportation.
- Drivers shall confirm cargo delivery before a booking is marked as completed.

6. Other Requirements

This section describes additional requirements that support the successful implementation and future expansion of the system.

6.1 Database Requirements

The system shall use **PostgreSQL** as the relational database management system.

The database shall maintain information related to:

- User Accounts
- Truck Providers
- Shippers
- Truck Routes
- Freight Bookings
- Capacity Information
- Transaction Records
- System Logs

All database tables shall be normalized to reduce redundancy and improve query performance. Primary keys and foreign keys shall be used to maintain referential integrity among related tables.

Automatic backup and recovery mechanisms shall be implemented to prevent data loss.

6.2 Internationalization Requirements

Initially, the application shall support the English language for all user interfaces.

Future versions may support additional regional languages such as:

- Tamil
- Hindi
- Telugu
- Kannada

Language selection shall be configurable without affecting system functionality.

6.3 Legal and Regulatory Requirements

The proposed system shall comply with applicable software development and data protection regulations.

These include:

- Information Technology Act, 2000 (India)
- Digital data privacy guidelines applicable to web applications
- Standard software security practices
- Secure storage of user information
- Protection of confidential business data

The application shall ensure that sensitive user information is protected against unauthorized access.

6.4 Reuse Objectives

The software shall be designed using reusable and modular components.

The following modules shall be reusable in future logistics or transportation applications:

- User Authentication Module
- Route Management Module
- Booking Management Module
- Notification Module
- Report Generation Module
- Search and Matching Module

The modular architecture will simplify maintenance and future feature additions.

6.5 Future Enhancements

The following enhancements may be incorporated in future versions of the system:

- Real-time GPS tracking of trucks
- Online payment gateway integration
- AI-based dynamic route optimization
- Machine learning for demand prediction
- Mobile applications for Android and iOS
- SMS and WhatsApp notifications
- Live traffic integration
- Automatic freight price optimization
- Driver verification using Aadhaar or Digital ID
- Analytics dashboard with predictive logistics insights

Appendix A: Glossary

Administrator

A privileged system user responsible for managing users, monitoring truck routes, supervising bookings, generating reports, resolving disputes, and maintaining the overall operation of the system.

AI Matching Engine

The intelligent software module responsible for comparing freight requests with registered truck return routes and identifying suitable transportation opportunities based on predefined route corridors and available cargo capacity.

Booking

A reservation made by a shipper to transport goods using the available cargo capacity of a registered truck.

Booking ID

A unique identification number automatically generated by the system for every successful booking transaction.

Cargo

Goods or materials transported from one location to another using trucks registered in the system.

Cargo Capacity

The maximum amount of cargo that a truck can carry during its return journey.

Driver

A registered user responsible for operating the assigned truck, updating trip status, confirming cargo pickup and delivery, and reporting trip completion.

Freight

Commercial goods transported from one location to another.

Freight Optimization

The process of efficiently allocating available truck capacity to reduce empty-return trips, transportation costs, and fuel consumption.

PostgreSQL

An open-source relational database management system used for storing all application data.

Route Corridor

A predefined sequence of cities representing the return path of a truck.

Route Matching

The process of comparing shipper requests with registered truck routes to identify suitable transportation opportunities.

Shipper

An individual or business organization that requires transportation services for shipping goods.

Truck Provider

A logistics company, fleet owner, or individual truck operator who registers return routes and offers available cargo capacity.

Trip Status

The current operational stage of a truck journey, such as Scheduled, In Transit, Delivered, or Completed.

User Authentication

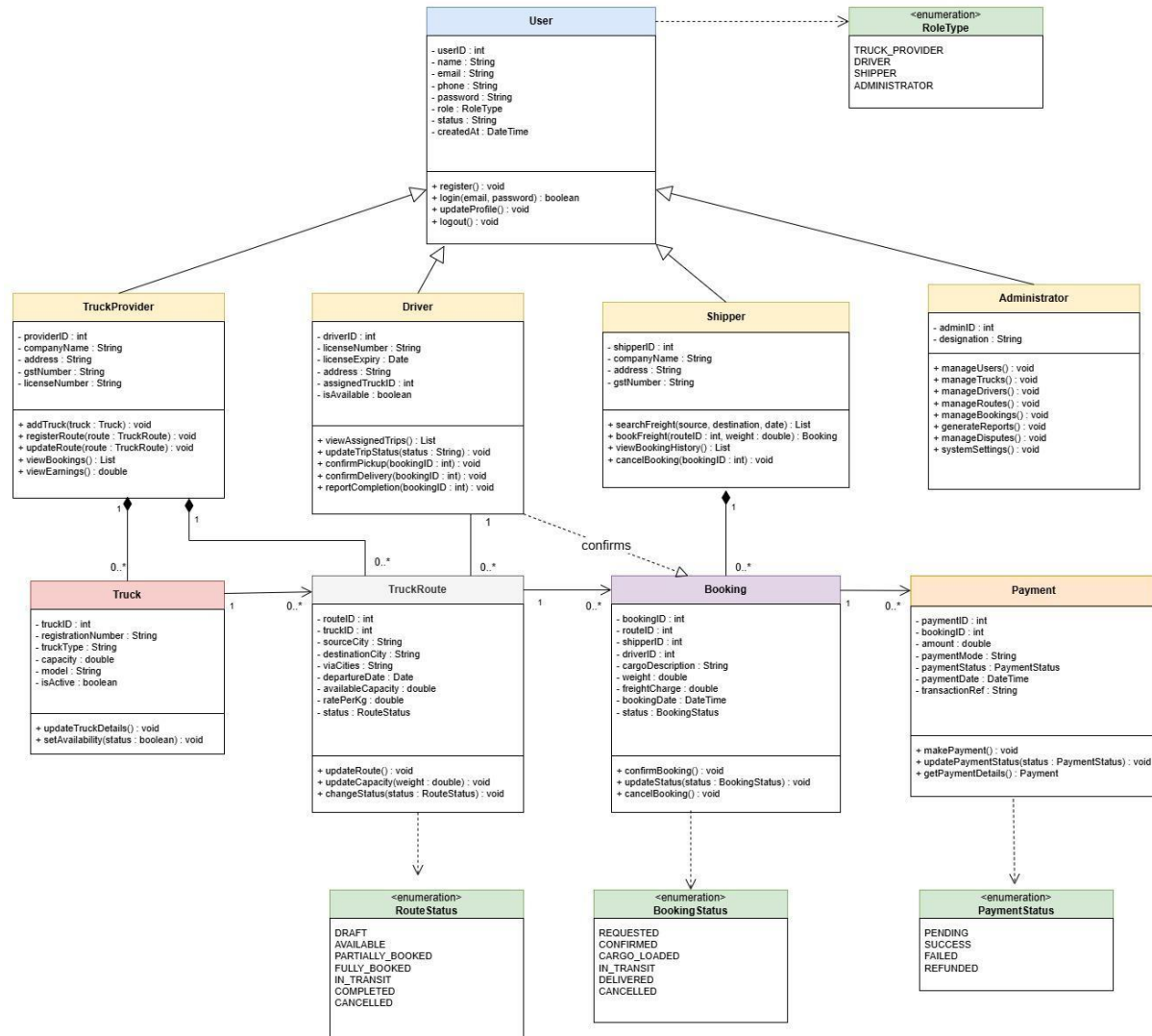
The process of verifying user identity before granting access to the application.

Vehicle Utilization

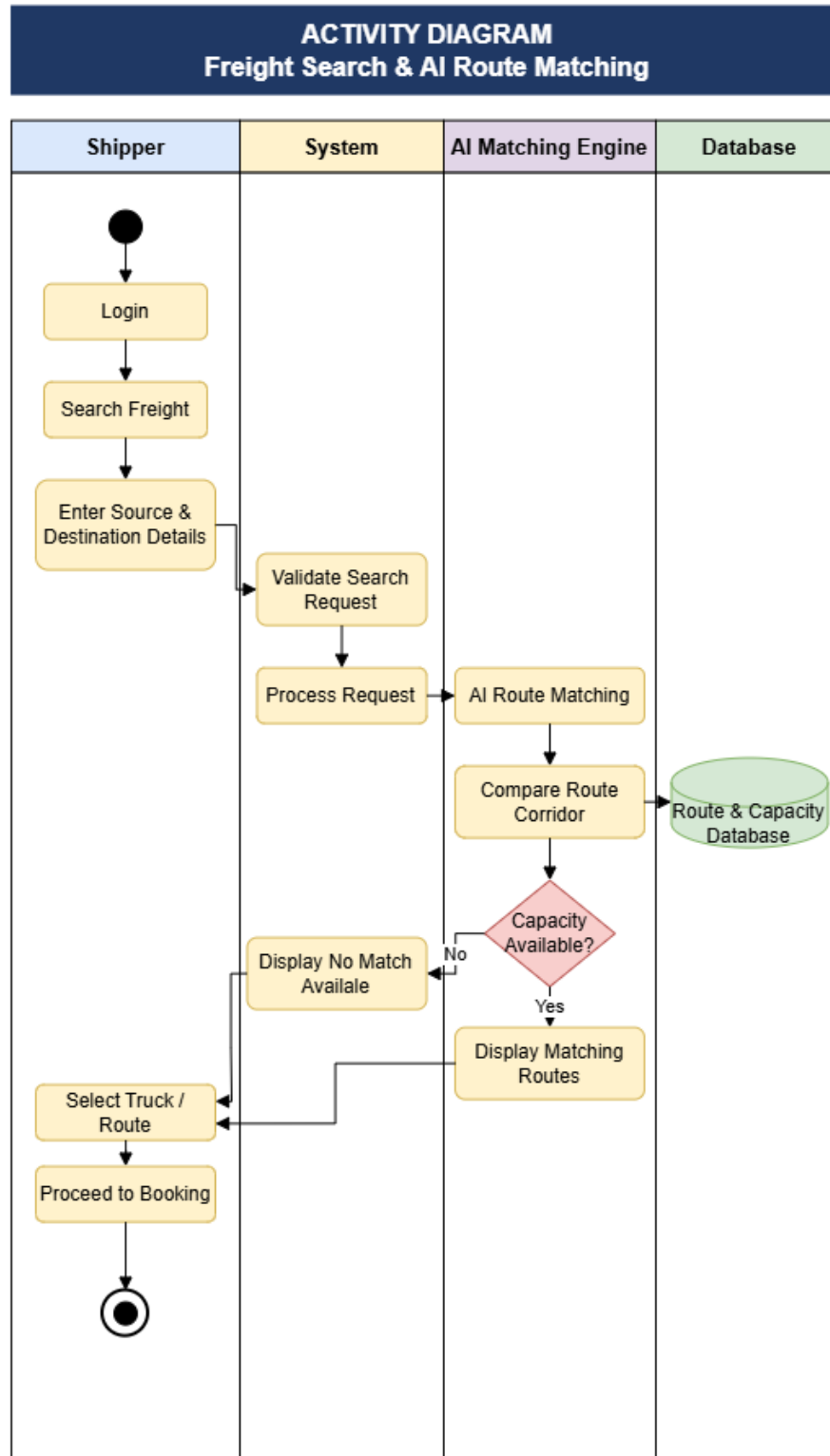
The effective use of truck cargo capacity to maximize transportation efficiency and reduce empty journeys.

Appendix B: Analysis Models

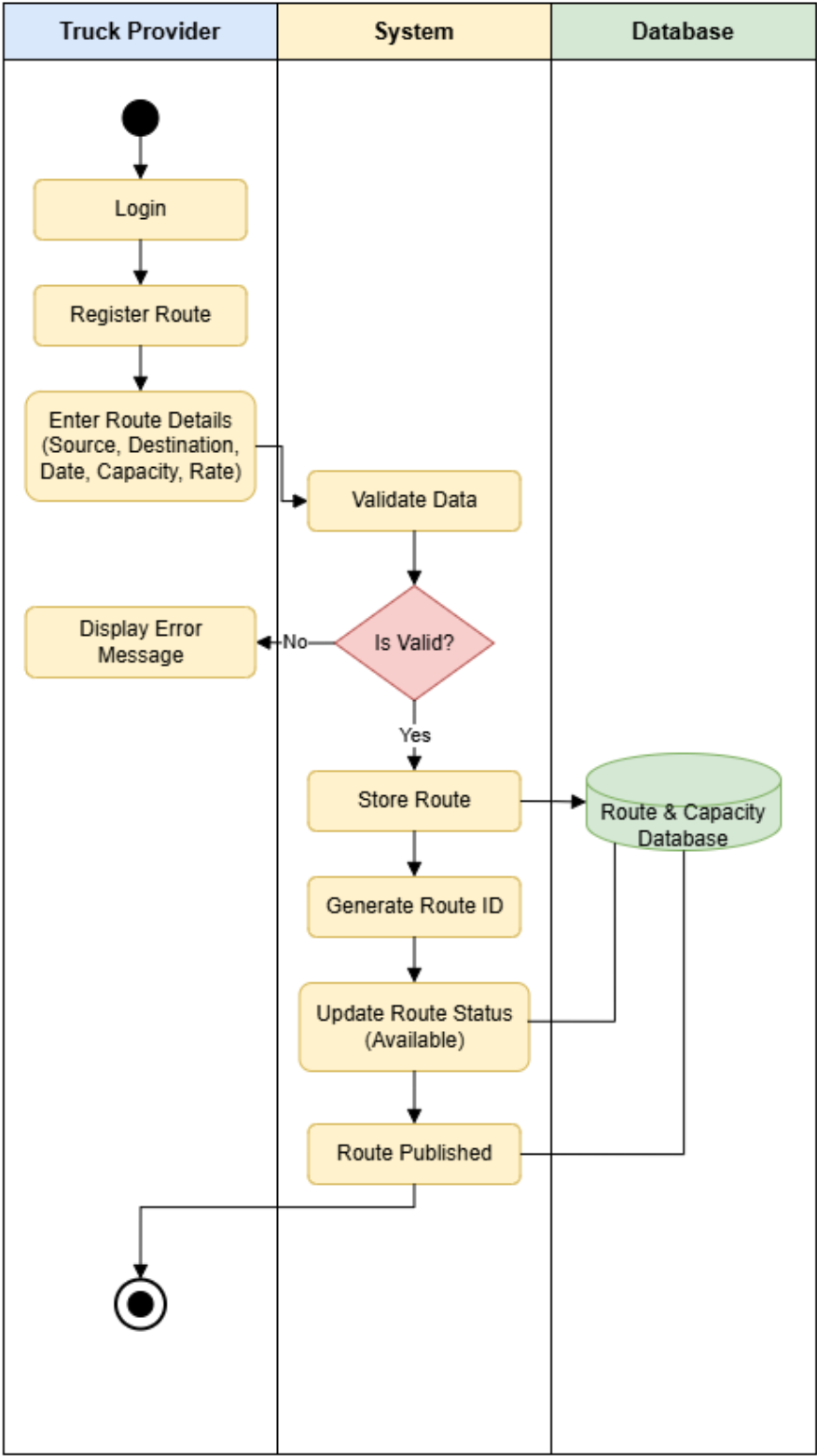
UML Class Diagram:



UML Activity Diagram:

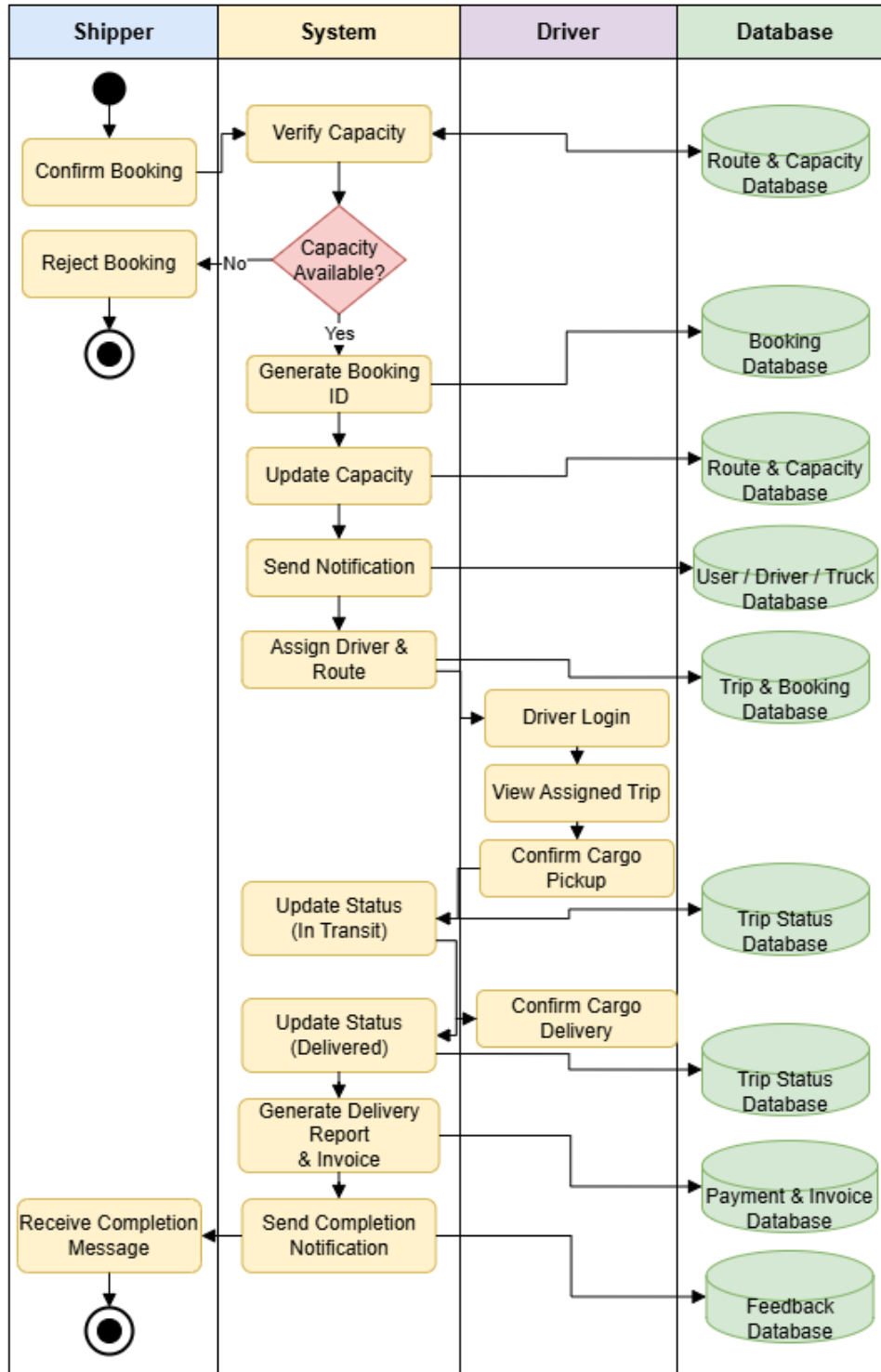


ACTIVITY DIAGRAM
Route Management



ACTIVITY DIAGRAM

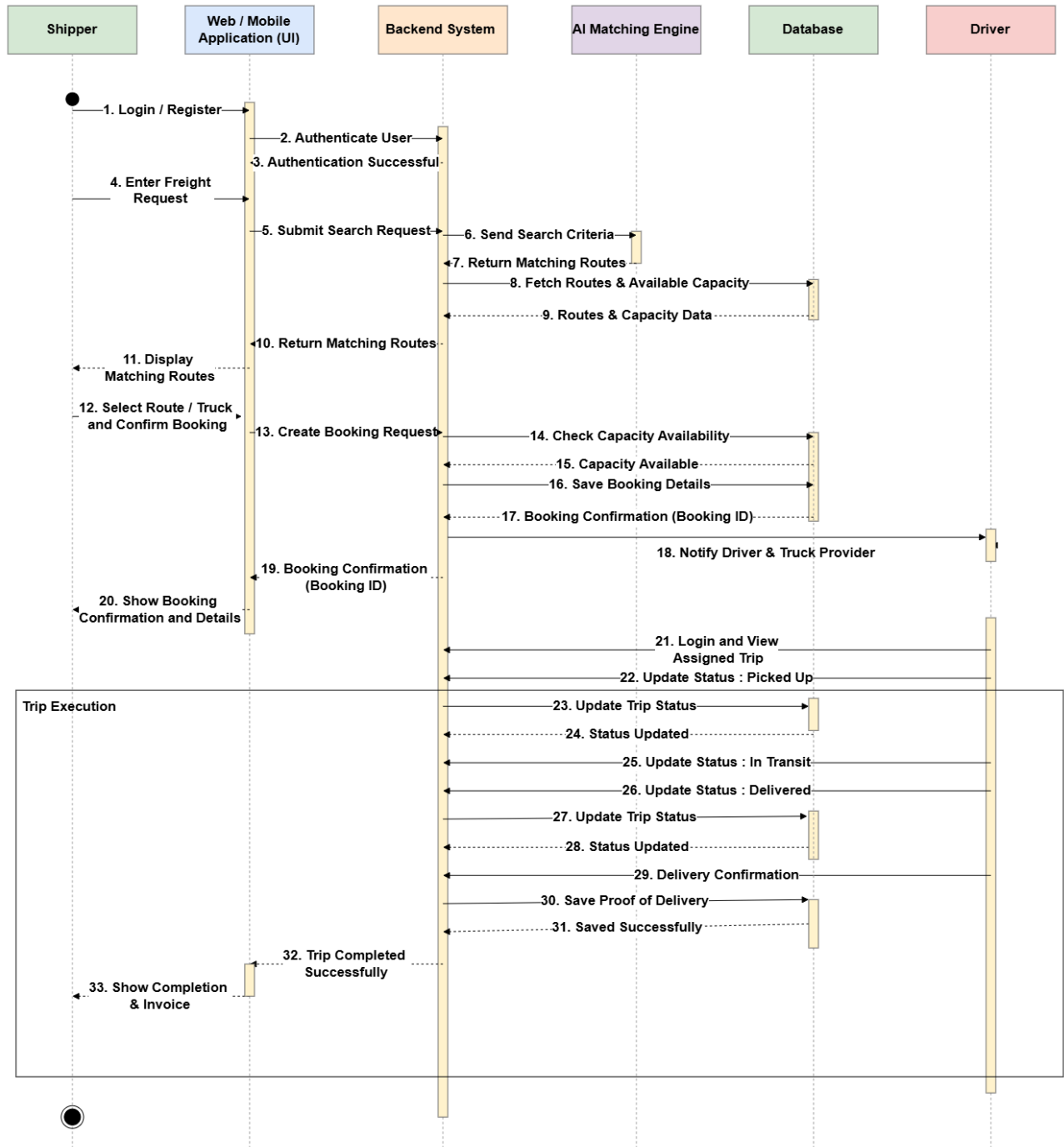
Booking, Capacity Allocation & Trip Management



UML Sequence Diagram:

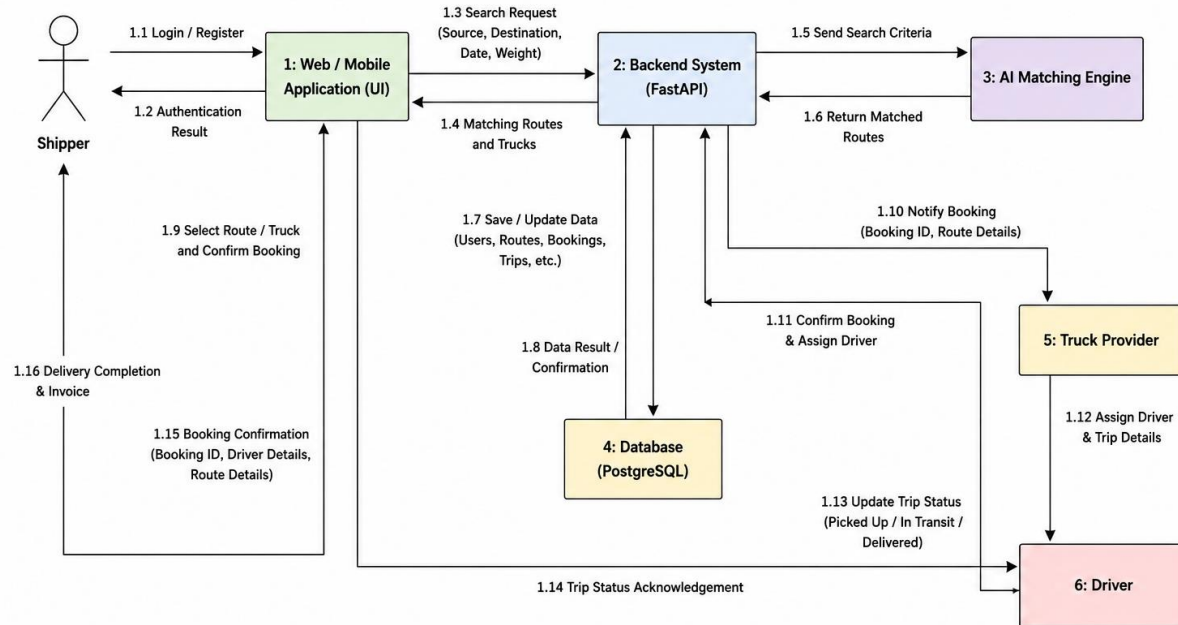
AI-Based Empty-Return Truck Sharing and Freight Optimization System

Sequence Diagram - Freight Search, Booking and Trip Completion



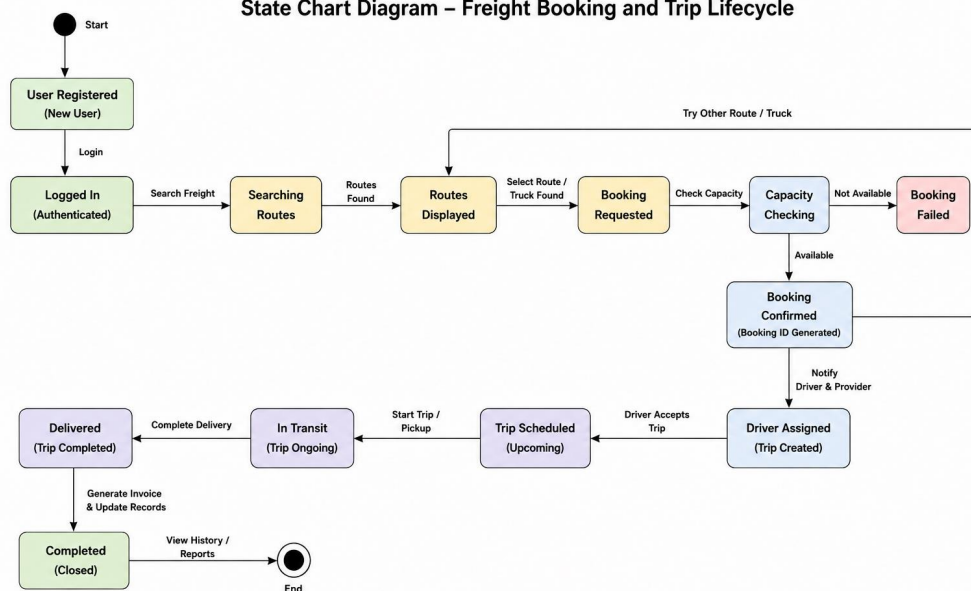
UML Collaboration Diagram:

Collaboration Diagram – Freight Search, Booking and Trip Execution



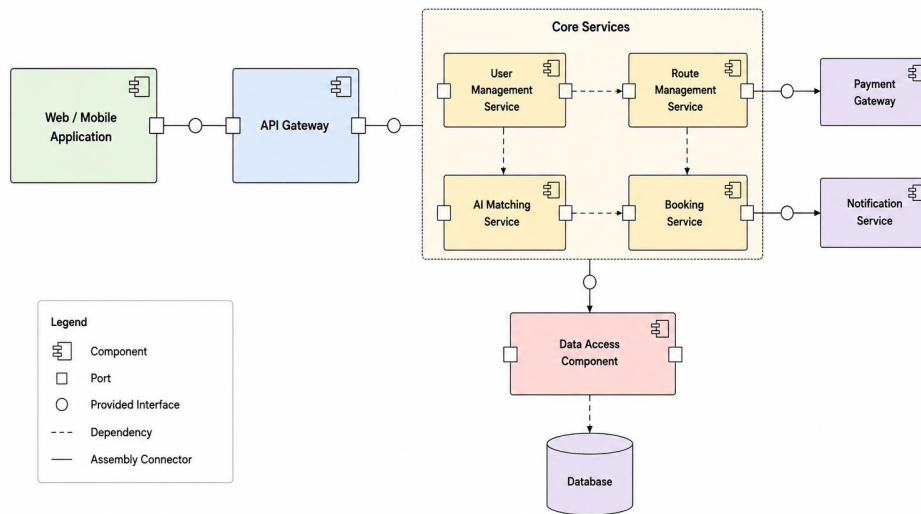
UML State Chart Diagram:

State Chart Diagram – Freight Booking and Trip Lifecycle



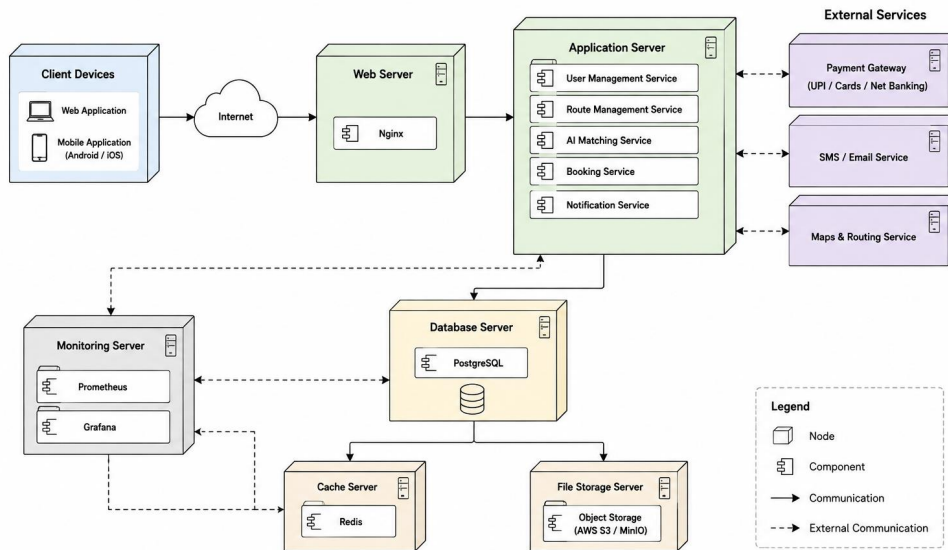
UML Component Diagram:

UML Component Diagram – Freight Booking and Optimization System



UML Deployment Diagram:

Deployment Diagram – AI-Based Empty-Return Truck Sharing and Freight Optimization System



Appendix C: Issues List

The following features are identified for future development and enhancement.

TBD-1: Integration of GPS-based real-time vehicle tracking.

TBD-2: Online payment gateway integration.

TBD-3: AI-based dynamic freight pricing.

TBD-4: SMS, Email, and WhatsApp notifications.

TBD-5: Android and iOS mobile applications.

TBD-6: Machine Learning-based demand forecasting.

TBD-7: Predictive route optimization using historical traffic data.

TBD-8: Integration with third-party logistics providers.

TBD-9: Multi-language user interface supporting English, Tamil, Hindi, Telugu, and Kannada.

TBD-10: Dashboard with advanced business intelligence and analytics.

Source Code

1. database

```
CREATE DATABASE freight_db;
CREATE USER freight_user WITH PASSWORD 'freight_pass';
GRANT ALL PRIVILEGES ON DATABASE freight_db TO freight_user;
-- For PostgreSQL 15+, also run:
\c freight_db
GRANT ALL ON SCHEMA public TO freight_user;

CREATE TYPE user_role AS ENUM ('truck_provider', 'shipper', 'driver', 'admin');
CREATE TYPE route_status AS ENUM ('active', 'completed', 'cancelled');
CREATE TYPE booking_status AS ENUM ('pending', 'confirmed', 'picked_up', 'delivered', 'cancelled');

CREATE TABLE users (
    id                VARCHAR(36) PRIMARY KEY,
    full_name         VARCHAR(150) NOT NULL,
    email             VARCHAR(150) NOT NULL UNIQUE,
    phone             VARCHAR(20),
    password_hash     VARCHAR(255) NOT NULL,
    role              user_role NOT NULL,
    company_name      VARCHAR(150),
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,
    failed_login_attempts INTEGER NOT NULL DEFAULT 0,
    locked_until      TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_users_email ON users (email);
CREATE INDEX idx_users_role ON users (role);

CREATE TABLE truck_routes (
    id                VARCHAR(36) PRIMARY KEY,
    route_code        VARCHAR(20) NOT NULL UNIQUE,
    provider_id       VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    driver_id         VARCHAR(36) REFERENCES users(id) ON DELETE SET NULL,
    source_city       VARCHAR(100) NOT NULL,
    destination_city  VARCHAR(100) NOT NULL,
    intermediate_hubs VARCHAR(255),
    return_date       TIMESTAMPTZ NOT NULL,
    total_capacity_kg DOUBLE PRECISION NOT NULL CHECK (total_capacity_kg > 0),
    available_capacity_kg DOUBLE PRECISION NOT NULL CHECK (available_capacity_kg >= 0),
    rate_per_kg       DOUBLE PRECISION NOT NULL CHECK (rate_per_kg > 0),
    status            route_status NOT NULL DEFAULT 'active',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CHECK (available_capacity_kg <= total_capacity_kg)
);
CREATE INDEX idx_routes_provider ON truck_routes (provider_id);
CREATE INDEX idx_routes_driver ON truck_routes (driver_id);
CREATE INDEX idx_routes_source ON truck_routes (source_city);
CREATE INDEX idx_routes_destination ON truck_routes (destination_city);
CREATE INDEX idx_routes_status ON truck_routes (status);

CREATE TABLE bookings (
    id                VARCHAR(36) PRIMARY KEY,
    booking_ref       VARCHAR(20) NOT NULL UNIQUE,
    route_id          VARCHAR(36) NOT NULL REFERENCES truck_routes(id) ON DELETE CASCADE,
    shipper_id        VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    pickup_location   VARCHAR(100) NOT NULL,
    delivery_location VARCHAR(100) NOT NULL,
    cargo_weight_kg   DOUBLE PRECISION NOT NULL CHECK (cargo_weight_kg > 0),
    cargo_volume_cbm  DOUBLE PRECISION CHECK (cargo_volume_cbm IS NULL OR cargo_volume_cbm >
0),
    cost              DOUBLE PRECISION NOT NULL CHECK (cost >= 0),
    status            booking_status NOT NULL DEFAULT 'confirmed',
    picked_up_at      TIMESTAMPTZ,
    delivered_at      TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

        updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );
CREATE INDEX idx_bookings_route ON bookings (route_id);
CREATE INDEX idx_bookings_shipper ON bookings (shipper_id);
CREATE INDEX idx_bookings_status ON bookings (status);

CREATE TABLE audit_logs (
    id          VARCHAR(36) PRIMARY KEY,
    user_id     VARCHAR(36) REFERENCES users(id) ON DELETE SET NULL,
    action      VARCHAR(100) NOT NULL,
    details     TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_audit_created ON audit_logs (created_at);
CREATE INDEX idx_audit_user ON audit_logs (user_id);

```

2. backend/app/config.py

```

import os
from dotenv import load_dotenv

load_dotenv()

class Settings:
    # Database
    DATABASE_URL: str = os.getenv(
        "DATABASE_URL",
        "postgresql://freight_user:freight_pass@localhost:5432/freight_db",
    )

    # JWT
    SECRET_KEY: str = os.getenv("SECRET_KEY", "CHANGE_ME_IN_PRODUCTION_SECRET_KEY")
    ALGORITHM: str = os.getenv("ALGORITHM", "HS256")
    ACCESS_TOKEN_EXPIRE_MINUTES: int = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "1440"))

    # CORS
    ALLOWED_ORIGINS: list = os.getenv("ALLOWED_ORIGINS", "http://localhost:5173").split(",")

    # Rate limiting
    RATE_LIMIT_PER_MINUTE: int = int(os.getenv("RATE_LIMIT_PER_MINUTE", "60"))

    # App
    APP_NAME: str = "AI-Based Empty-Return Truck Sharing and Freight Optimization System"
    ENV: str = os.getenv("ENV", "development")

settings = Settings()

```

3. backend/app/database.py

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

from app.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    """FastAPI dependency that yields a database session and ensures it's closed."""
    db = SessionLocal()

```

```

try:
    yield db
finally:
    db.close()

```

4. backend/app/models.py

```

import enum
import uuid

from sqlalchemy import (
    Column,
    String,
    Integer,
    Float,
    Boolean,
    DateTime,
    ForeignKey,
    Enum,
    Text,
    func,
)
from sqlalchemy.orm import relationship

from app.database import Base

def gen_uuid() -> str:
    return str(uuid.uuid4())

class UserRole(str, enum.Enum):
    truck_provider = "truck_provider"
    shipper = "shipper"
    driver = "driver"
    admin = "admin"

class RouteStatus(str, enum.Enum):
    active = "active"
    completed = "completed"
    cancelled = "cancelled"

class BookingStatus(str, enum.Enum):
    pending = "pending"
    confirmed = "confirmed"
    picked_up = "picked_up"
    delivered = "delivered"
    cancelled = "cancelled"

class User(Base):
    __tablename__ = "users"

    id = Column(String(36), primary_key=True, default=gen_uuid)
    full_name = Column(String(150), nullable=False)
    email = Column(String(150), unique=True, nullable=False, index=True)
    phone = Column(String(20), nullable=True)
    password_hash = Column(String(255), nullable=False)
    role = Column(Enum(UserRole), nullable=False, index=True)
    company_name = Column(String(150), nullable=True)
    is_active = Column(Boolean, default=True, nullable=False)
    failed_login_attempts = Column(Integer, default=0, nullable=False)
    locked_until = Column(DateTime(timezone=True), nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
onupdate=func.now())

```

```

    routes = relationship("TruckRoute", back_populates="provider",
foreign_keys="TruckRoute.provider_id")
    driven_routes = relationship("TruckRoute", back_populates="driver",
foreign_keys="TruckRoute.driver_id")
    bookings = relationship("Booking", back_populates="shipper",
foreign_keys="Booking.shipper_id")

```

```

class Vehicle(Base):
    __tablename__ = "vehicles"
    id = Column(String(36), primary_key=True, default=gen_uuid)
    provider_id = Column(String(36), ForeignKey("users.id"), nullable=False, index=True)
    vehicle_number = Column(String(20), unique=True, nullable=False)
    vehicle_type = Column(String(50), nullable=False) # Truck, Mini Truck, Trailer
    capacity_kg = Column(Float, nullable=False)
    is_active = Column(Boolean, default=True, nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    provider = relationship("User", backref="vehicles")
    routes = relationship("TruckRoute", back_populates="vehicle")

```

```

class TruckRoute(Base):
    __tablename__ = "truck_routes"

    id = Column(String(36), primary_key=True, default=gen_uuid)
    route_code = Column(String(20), unique=True, nullable=False, index=True)
    provider_id = Column(String(36), ForeignKey("users.id"), nullable=False, index=True)
    driver_id = Column(String(36), ForeignKey("users.id"), nullable=True, index=True)
    vehicle_id = Column(String(36), ForeignKey("vehicles.id"), nullable=True, index=True)

    source_city = Column(String(100), nullable=False, index=True)
    destination_city = Column(String(100), nullable=False, index=True)
    intermediate_hubs = Column(String(255), nullable=True) # comma-separated
    return_date = Column(DateTime(timezone=True), nullable=False)

    total_capacity_kg = Column(Float, nullable=False)
    available_capacity_kg = Column(Float, nullable=False)
    rate_per_kg = Column(Float, nullable=False)

    status = Column(Enum(RouteStatus), default=RouteStatus.active, nullable=False, index=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
onupdate=func.now())

    provider = relationship("User", back_populates="routes", foreign_keys=[provider_id])
    driver = relationship("User", back_populates="driven_routes", foreign_keys=[driver_id])
    vehicle = relationship("Vehicle", back_populates="routes")
    bookings = relationship("Booking", back_populates="route", cascade="all, delete-orphan")

```

```

class Booking(Base):
    __tablename__ = "bookings"

    id = Column(String(36), primary_key=True, default=gen_uuid)
    booking_ref = Column(String(20), unique=True, nullable=False, index=True)
    route_id = Column(String(36), ForeignKey("truck_routes.id"), nullable=False, index=True)
    shipper_id = Column(String(36), ForeignKey("users.id"), nullable=False, index=True)

    pickup_location = Column(String(100), nullable=False)
    delivery_location = Column(String(100), nullable=False)
    cargo_weight_kg = Column(Float, nullable=False)
    cargo_volume_cbm = Column(Float, nullable=True)

    cost = Column(Float, nullable=False)
    status = Column(Enum(BookingStatus), default=BookingStatus.confirmed, nullable=False,
index=True)

    picked_up_at = Column(DateTime(timezone=True), nullable=True)
    delivered_at = Column(DateTime(timezone=True), nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

```

```

    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
onupdate=func.now())

    route = relationship("TruckRoute", back_populates="bookings")
    shipper = relationship("User", back_populates="bookings", foreign_keys=[shipper_id])

class AuditLog(Base):
    __tablename__ = "audit_logs"

    id = Column(String(36), primary_key=True, default=gen_uuid)
    user_id = Column(String(36), ForeignKey("users.id"), nullable=True, index=True)
    action = Column(String(100), nullable=False)
    details = Column(Text, nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now(), index=True)

```

5. backend/app/schemas.py

```

import re
from datetime import datetime
from typing import Optional, List

from pydantic import BaseModel, EmailStr, Field, field_validator

from app.models import UserRole, RouteStatus, BookingStatus

# ----- Auth / Users -----

class UserRegister(BaseModel):
    full_name: str = Field(..., min_length=2, max_length=150)
    email: EmailStr
    phone: str = Field(..., min_length=7, max_length=20)
    password: str = Field(..., min_length=8, max_length=128)
    role: UserRole
    company_name: Optional[str] = Field(None, max_length=150)

    @field_validator("role")
    @classmethod
    def restrict_public_roles(cls, v: UserRole) -> UserRole:
        # Admin accounts must not be self-registered via the public endpoint.
        if v == UserRole.admin:
            raise ValueError("Admin accounts cannot be created via public registration")
        return v

    @field_validator("password")
    @classmethod
    def password_strength(cls, v: str) -> str:
        if not re.search(r"[A-Z]", v):
            raise ValueError("Password must contain at least one uppercase letter")
        if not re.search(r"[a-z]", v):
            raise ValueError("Password must contain at least one lowercase letter")
        if not re.search(r"\d", v):
            raise ValueError("Password must contain at least one digit")
        return v

    @field_validator("phone")
    @classmethod
    def phone_format(cls, v: str) -> str:
        if not re.fullmatch(r"\+?[0-9\-\s]{7,20}", v):
            raise ValueError("Invalid phone number format")
        return v

class UserLogin(BaseModel):
    email: EmailStr
    password: str

```

```

class UserOut(BaseModel):
    id: str
    full_name: str
    email: EmailStr
    phone: Optional[str]
    role: UserRole
    company_name: Optional[str]
    is_active: bool
    created_at: datetime

    class Config:
        from_attributes = True

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
    user: UserOut

# ----- Vehicles -----
class VehicleCreate(BaseModel):
    vehicle_number: str = Field(..., min_length=2, max_length=20)
    vehicle_type: str = Field(..., min_length=2, max_length=50)
    capacity_kg: float = Field(..., gt=0)

class VehicleOut(BaseModel):
    id: str
    provider_id: str
    vehicle_number: str
    vehicle_type: str
    capacity_kg: float
    is_active: bool
    created_at: datetime
    class Config:
        from_attributes = True

# ----- Truck Routes -----

class TruckRouteCreate(BaseModel):
    source_city: str = Field(..., min_length=2, max_length=100)
    destination_city: str = Field(..., min_length=2, max_length=100)
    intermediate_hubs: Optional[str] = Field(None, max_length=255)
    return_date: datetime
    total_capacity_kg: float = Field(..., gt=0)
    rate_per_kg: float = Field(..., gt=0)
    vehicle_id: Optional[str] = None

    @field_validator("destination_city")
    @classmethod
    def different_cities(cls, v, info):
        source = info.data.get("source_city")
        if source and v.strip().lower() == source.strip().lower():
            raise ValueError("Source and destination cities must be different")
        return v

class TruckRouteUpdate(BaseModel):
    source_city: Optional[str] = Field(None, min_length=2, max_length=100)
    destination_city: Optional[str] = Field(None, min_length=2, max_length=100)
    intermediate_hubs: Optional[str] = Field(None, max_length=255)
    return_date: Optional[datetime] = None
    total_capacity_kg: Optional[float] = Field(None, gt=0)
    rate_per_kg: Optional[float] = Field(None, gt=0)
    status: Optional[RouteStatus] = None

class TruckRouteOut(BaseModel):
    id: str
    route_code: str

```

```

    provider_id: str
    driver_id: Optional[str]
    vehicle_id: Optional[str] = None
    source_city: str
    destination_city: str
    intermediate_hubs: Optional[str]
    return_date: datetime
    total_capacity_kg: float
    available_capacity_kg: float
    rate_per_kg: float
    status: RouteStatus
    created_at: datetime

    class Config:
        from_attributes = True

class DriverAssign(BaseModel):
    driver_id: str

# ----- Search -----

class SearchResult(BaseModel):
    route: TruckRouteOut
    provider_name: str
    estimated_cost_for_query: Optional[float] = None

# ----- Bookings -----

class BookingCreate(BaseModel):
    route_id: str
    pickup_location: str = Field(..., min_length=2, max_length=100)
    delivery_location: str = Field(..., min_length=2, max_length=100)
    cargo_weight_kg: float = Field(..., gt=0)
    cargo_volume_cbm: Optional[float] = Field(None, gt=0)

class BookingOut(BaseModel):
    id: str
    booking_ref: str
    route_id: str
    shipper_id: str
    pickup_location: str
    delivery_location: str
    cargo_weight_kg: float
    cargo_volume_cbm: Optional[float]
    cost: float
    status: BookingStatus
    picked_up_at: Optional[datetime]
    delivered_at: Optional[datetime]
    created_at: datetime

    class Config:
        from_attributes = True

class BookingStatusUpdate(BaseModel):
    status: BookingStatus

# ----- Admin / Reports -----

class ReportSummary(BaseModel):
    total_users: int
    total_providers: int
    total_shippers: int
    total_drivers: int
    total_routes: int

```

```
active_routes: int
completed_routes: int
total_bookings: int
total_revenue: float
average_capacity_utilization_pct: float
```

```
class PaginatedResponse(BaseModel):
    items: List[dict]
    total: int
    page: int
    page_size: int
```

6. backend/app/security.py

```
from datetime import datetime, timedelta, timezone
from typing import Optional

from jose import jwt, JWTError
from passlib.context import CryptContext

from app.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (
        expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

def decode_access_token(token: str) -> Optional[dict]:
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        return payload
    except JWTError:
        return None
```

7. backend/app/deps.py

```
from typing import List

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from app.database import get_db
from app.models import User, UserRole
from app.security import decode_access_token

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/auth/login")

def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
```

```

        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    payload = decode_access_token(token)
    if payload is None:
        raise credentials_exception

    user_id: str = payload.get("sub")
    if user_id is None:
        raise credentials_exception

    user = db.query(User).filter(User.id == user_id).first()
    if user is None:
        raise credentials_exception
    if not user.is_active:
        raise HTTPException(status_code=403, detail="Account is deactivated")
    return user

def require_roles(*allowed_roles: UserRole):
    """Dependency factory enforcing Role-Based Access Control (RBAC)."""

    def _checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="You do not have permission to perform this action",
            )
        return current_user

    return _checker

```

8. backend/app/main.py

```

import time
from collections import defaultdict, deque

from fastapi import FastAPI, Request, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

from app.config import settings
from app.database import Base, engine
from app.routers import auth, routes, search, bookings, drivers, admin, vehicles

# Create database tables (use Alembic migrations in production instead).
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title=settings.APP_NAME,
    description="REST API for the AI-Based Empty-Return Truck Sharing and Freight Optimization System.",
    version="1.0.0",
)

# ----- CORS -----
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ----- Security headers (Helmet-equivalent) -----
@app.middleware("http")
async def add_security_headers(request: Request, call_next):

```

```

response = await call_next(request)
response.headers["X-Content-Type-Options"] = "nosniff"
response.headers["X-Frame-Options"] = "DENY"
response.headers["X-XSS-Protection"] = "1; mode=block"
response.headers["Strict-Transport-Security"] = "max-age=63072000; includeSubDomains"
response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
return response

# ----- Simple in-memory rate limiting (per client IP) -----
_request_log: dict = defaultdict(deque)

@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next):
    client_ip = request.client.host if request.client else "unknown"
    now = time.time()
    window = 60.0
    log = _request_log[client_ip]

    while log and now - log[0] > window:
        log.popleft()

    if len(log) >= settings.RATE_LIMIT_PER_MINUTE:
        return JSONResponse(
            status_code=status.HTTP_429_TOO_MANY_REQUESTS,
            content={"detail": "Too many requests. Please slow down and try again shortly."},
        )

    log.append(now)
    return await call_next(request)

# ----- Validation error handler (clean, non-leaky error messages) -----
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    errors = [{"field": ".".join(str(x) for x in e["loc"][1:]), "message": e["msg"]} for e in exc.errors()]
    return JSONResponse(status_code=422, content={"detail": errors})

# ----- Routers -----
app.include_router(auth.router)
app.include_router(routes.router)
app.include_router(search.router)
app.include_router(bookings.router)
app.include_router(drivers.router)
app.include_router(admin.router)
app.include_router(vehicles.router)

@app.get("/api/health", tags=["System"])
def health_check():
    return {"status": "ok", "service": settings.APP_NAME, "environment": settings.ENV}

```

9. backend/app/routers/auth.py

```

from datetime import datetime, timedelta, timezone

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.deps import get_current_user
from app.models import User, AuditLog
from app.schemas import UserRegister, UserLogin, UserOut, Token
from app.security import hash_password, verify_password, create_access_token

router = APIRouter(prefix="/api/auth", tags=["Authentication"])

```

```
MAX_FAILED_ATTEMPTS = 5
LOCKOUT_MINUTES = 15
```

```
@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(payload: UserRegister, db: Session = Depends(get_db)):
    """FR-1, FR-2, FR-3, FR-4: Register a Truck Provider, Shipper, or Driver."""
    existing = db.query(User).filter(User.email == payload.email.lower()).first()
    if existing:
        raise HTTPException(status_code=400, detail="An account with this email already exists")

    user = User(
        full_name=payload.full_name.strip(),
        email=payload.email.lower(),
        phone=payload.phone,
        password_hash=hash_password(payload.password),
        role=payload.role,
        company_name=payload.company_name,
    )
    db.add(user)
    db.commit()
    db.refresh(user)

    db.add(AuditLog(user_id=user.id, action="REGISTER", details=f"New {user.role.value} account created"))
    db.commit()

    return user
```

```
@router.post("/login", response_model=Token)
def login(payload: UserLogin, db: Session = Depends(get_db)):
    """FR-5, FR-6: Authenticate user and issue JWT. Includes brute-force logout."""
    user = db.query(User).filter(User.email == payload.email.lower()).first()

    if user and user.locked_until and user.locked_until > datetime.now(timezone.utc):
        raise HTTPException(
            status_code=status.HTTP_423_LOCKED,
            detail=f"Account locked due to repeated failed logins. Try again after {user.locked_until.isoformat()}",
        )

    if not user or not verify_password(payload.password, user.password_hash):
        if user:
            user.failed_login_attempts += 1
            if user.failed_login_attempts >= MAX_FAILED_ATTEMPTS:
                user.locked_until = datetime.now(timezone.utc) +
timedelta(minutes=LOCKOUT_MINUTES)
                db.add(AuditLog(user_id=user.id, action="ACCOUNT_LOCKED", details="Too many failed login attempts"))
                db.commit()
                raise HTTPException(status_code=401, detail="Incorrect email or password")

        if not user.is_active:
            raise HTTPException(status_code=403, detail="Account is deactivated. Contact an administrator.")

    user.failed_login_attempts = 0
    user.locked_until = None
    db.add(AuditLog(user_id=user.id, action="LOGIN", details="Successful login"))
    db.commit()

    access_token = create_access_token(data={"sub": user.id, "role": user.role.value})
    return Token(access_token=access_token, user=user)
```

```
@router.get("/me", response_model=UserOut)
def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

10. backend/app/routers/routes.py

```
import random
import string
from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session

from app.database import get_db
from app.deps import get_current_user, require_roles
from app.models import User, UserRole, TruckRoute, RouteStatus, AuditLog
from app.schemas import TruckRouteCreate, TruckRouteUpdate, TruckRouteOut, DriverAssign

router = APIRouter(prefix="/api/routes", tags=["Truck Routes"])

def generate_route_code() -> str:
    return "RT-" + "".join(random.choices(string.ascii_uppercase + string.digits, k=8))

@router.post("", response_model=TruckRouteOut, status_code=201)
def create_route(
    payload: TruckRouteCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider)),
):
    """FR-7, FR-8, FR-9: Truck Provider registers a return route with capacity."""
    code = generate_route_code()
    while db.query(TruckRoute).filter(TruckRoute.route_code == code).first():
        code = generate_route_code()

    vehicle_id = None
    if payload.vehicle_id:
        from app.models import Vehicle
        vehicle = db.query(Vehicle).filter(Vehicle.id == payload.vehicle_id,
        Vehicle.provider_id == current_user.id).first()
        if not vehicle:
            raise HTTPException(status_code=404, detail="Vehicle not found or does not belong
to you")
        vehicle_id = vehicle.id

    route = TruckRoute(
        route_code=code,
        provider_id=current_user.id,
        vehicle_id=vehicle_id,
        source_city=payload.source_city.strip(),
        destination_city=payload.destination_city.strip(),
        intermediate_hubs=payload.intermediate_hubs,
        return_date=payload.return_date,
        total_capacity_kg=payload.total_capacity_kg,
        available_capacity_kg=payload.total_capacity_kg,
        rate_per_kg=payload.rate_per_kg,
        status=RouteStatus.active,
    )
    db.add(route)
    db.commit()
    db.refresh(route)

    db.add(AuditLog(user_id=current_user.id, action="ROUTE_CREATED", details=f"Route {code}
created"))
    db.commit()
    return route

@router.get("/my", response_model=List[TruckRouteOut])
def my_routes(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider)),
):
    """FR-11: Display all active routes in the provider dashboard."""
```

```

    return (
        db.query(TruckRoute)
        .filter(TruckRoute.provider_id == current_user.id)
        .order_by(TruckRoute.created_at.desc())
        .all()
    )

@router.get("/available-drivers", response_model=list)
def list_drivers(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider, UserRole.admin)),
):
    drivers = db.query(User).filter(User.role == UserRole.driver, User.is_active == True).all()
    return [{"id": d.id, "full_name": d.full_name, "email": d.email, "phone": d.phone} for d in drivers]

@router.get("/{route_id}", response_model=TruckRouteOut)
def get_route(route_id: str, db: Session = Depends(get_db), current_user: User =
Depends(get_current_user)):
    route = db.query(TruckRoute).filter(TruckRoute.id == route_id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Route not found")
    return route

@router.put("/{route_id}", response_model=TruckRouteOut)
def update_route(
    route_id: str,
    payload: TruckRouteUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider, UserRole.admin)),
):
    """FR-10: Providers may edit registered routes before departure."""
    route = db.query(TruckRoute).filter(TruckRoute.id == route_id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Route not found")
    if current_user.role == UserRole.truck_provider and route.provider_id != current_user.id:
        raise HTTPException(status_code=403, detail="You do not own this route")
    if route.status == RouteStatus.completed:
        raise HTTPException(status_code=400, detail="Completed routes cannot be modified")

    data = payload.model_dump(exclude_unset=True)
    capacity_delta = None
    if "total_capacity_kg" in data:
        booked = route.total_capacity_kg - route.available_capacity_kg
        if data["total_capacity_kg"] < booked:
            raise HTTPException(
                status_code=400,
                detail="New capacity cannot be less than already-booked cargo weight",
            )
        capacity_delta = data["total_capacity_kg"] - route.total_capacity_kg

    for field, value in data.items():
        setattr(route, field, value)

    if capacity_delta is not None:
        route.available_capacity_kg += capacity_delta

    db.commit()
    db.refresh(route)
    return route

@router.delete("/{route_id}", status_code=204)
def delete_route(
    route_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider, UserRole.admin)),
):
    """FR-10: Providers may delete registered routes before departure."""

```

```

route = db.query(TruckRoute).filter(TruckRoute.id == route_id).first()
if not route:
    raise HTTPException(status_code=404, detail="Route not found")
if current_user.role == UserRole.truck_provider and route.provider_id != current_user.id:
    raise HTTPException(status_code=403, detail="You do not own this route")
if route.available_capacity_kg < route.total_capacity_kg:
    raise HTTPException(status_code=400, detail="Cannot delete a route with active
bookings")

db.delete(route)
db.commit()
return None

@router.patch("/{route_id}/assign-driver", response_model=TruckRouteOut)
def assign_driver(
    route_id: str,
    payload: DriverAssign,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider, UserRole.admin)),
):
    route = db.query(TruckRoute).filter(TruckRoute.id == route_id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Route not found")
    if current_user.role == UserRole.truck_provider and route.provider_id != current_user.id:
        raise HTTPException(status_code=403, detail="You do not own this route")

    driver = db.query(User).filter(User.id == payload.driver_id, User.role ==
UserRole.driver).first()
    if not driver:
        raise HTTPException(status_code=404, detail="Driver not found")

    route.driver_id = driver.id
    db.commit()
    db.refresh(route)
    return route

```

11. backend/app/routers/bookings.py

```

import io
import random
import string
from typing import List

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import StreamingResponse
from fpdf import FPDF
from sqlalchemy.orm import Session

from app.database import get_db
from app.deps import require_roles, get_current_user
from app.models import User, UserRole, TruckRoute, Booking, RouteStatus, BookingStatus,
AuditLog
from app.schemas import BookingCreate, BookingOut

router = APIRouter(prefix="/api/bookings", tags=["Bookings"])

def generate_booking_ref() -> str:
    return "BK-" + "".join(random.choices(string.digits + string.ascii_uppercase, k=8))

@router.post("", response_model=BookingOut, status_code=201)
def create_booking(
    payload: BookingCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.shipper)),
):
    """
    FR-17, FR-18, FR-20, FR-21: Reserve cargo space on a matched truck.

```

```

Generates a unique Booking ID, deducts booked weight from available
capacity, and prevents overbooking (BR: truck capacity shall never
become negative; a "Completed" route no longer accepts bookings).
"""
route = db.query(TruckRoute).filter(TruckRoute.id ==
payload.route_id).with_for_update().first()
if not route:
    raise HTTPException(status_code=404, detail="Route not found")
if route.status != RouteStatus.active:
    raise HTTPException(status_code=400, detail="This route is no longer accepting
bookings")
if payload.cargo_weight_kg > route.available_capacity_kg:
    raise HTTPException(status_code=400, detail="Insufficient available capacity on this
route")

ref = generate_booking_ref()
while db.query(Booking).filter(Booking.booking_ref == ref).first():
    ref = generate_booking_ref()

cost = round(payload.cargo_weight_kg * route.rate_per_kg, 2)

booking = Booking(
    booking_ref=ref,
    route_id=route.id,
    shipper_id=current_user.id,
    pickup_location=payload.pickup_location,
    delivery_location=payload.delivery_location,
    cargo_weight_kg=payload.cargo_weight_kg,
    cargo_volume_cbm=payload.cargo_volume_cbm,
    cost=cost,
    status=BookingStatus.confirmed,
)

# FR-18: deduct booked cargo weight from available capacity (never negative).
route.available_capacity_kg = round(route.available_capacity_kg - payload.cargo_weight_kg,
3)
if route.available_capacity_kg <= 0:
    route.available_capacity_kg = 0

db.add(booking)
db.add(AuditLog(user_id=current_user.id, action="BOOKING_CREATED", details=f"Booking {ref}
on route {route.route_code}"))
db.commit()
db.refresh(booking)
return booking

@router.get("/my", response_model=List[BookingOut])
def my_bookings(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.shipper)),
):
    """FR-19: Maintain booking history for Shippers."""
    return (
        db.query(Booking)
        .filter(Booking.shipper_id == current_user.id)
        .order_by(Booking.created_at.desc())
        .all()
    )

@router.get("/provider", response_model=List[BookingOut])
def provider_bookings(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.truck_provider)),
):
    """FR-19: Maintain booking history for Truck Providers, across their routes."""
    return (
        db.query(Booking)
        .join(TruckRoute, Booking.route_id == TruckRoute.id)
        .filter(TruckRoute.provider_id == current_user.id)
        .order_by(Booking.created_at.desc())
        .all()
    )

```

```

)

@router.get("/{booking_id}", response_model=BookingOut)
def get_booking(booking_id: str, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Booking not found")

    is_owner_shipper = current_user.role == UserRole.shipper and booking.shipper_id == current_user.id
    is_owner_provider = current_user.role == UserRole.truck_provider and booking.route.provider_id == current_user.id
    is_admin = current_user.role == UserRole.admin
    if not (is_owner_shipper or is_owner_provider or is_admin):
        raise HTTPException(status_code=403, detail="You do not have access to this booking")

    return booking

@router.patch("/{booking_id}/cancel", response_model=BookingOut)
def cancel_booking(
    booking_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.shipper, UserRole.admin)),
):
    booking = db.query(Booking).filter(Booking.id == booking_id).with_for_update().first()
    if not booking:
        raise HTTPException(status_code=404, detail="Booking not found")
    if current_user.role == UserRole.shipper and booking.shipper_id != current_user.id:
        raise HTTPException(status_code=403, detail="You do not own this booking")
    if booking.status in (BookingStatus.delivered, BookingStatus.cancelled):
        raise HTTPException(status_code=400, detail=f"Cannot cancel a booking that is already {booking.status.value}")

    route = booking.route
    route.available_capacity_kg = min(
        route.total_capacity_kg, round(route.available_capacity_kg + booking.cargo_weight_kg,
3)
    )
    booking.status = BookingStatus.cancelled

    db.add(AuditLog(user_id=current_user.id, action="BOOKING_CANCELLED", details=f"Booking {booking.booking_ref} cancelled"))
    db.commit()
    db.refresh(booking)
    return booking

@router.get("/{booking_id}/invoice")
def download_invoice(
    booking_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Generate and download a PDF invoice for a booking."""
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Booking not found")

    route = booking.route
    shipper = db.query(User).filter(User.id == booking.shipper_id).first()
    provider = db.query(User).filter(User.id == route.provider_id).first()

    # Access check: shipper, provider, assigned driver, or admin
    is_shipper = current_user.role == UserRole.shipper and booking.shipper_id == current_user.id
    is_provider = current_user.role == UserRole.truck_provider and route.provider_id == current_user.id
    is_driver = current_user.role == UserRole.driver and route.driver_id == current_user.id
    is_admin = current_user.role == UserRole.admin

```

```

if not (is_shipper or is_provider or is_driver or is_admin):
    raise HTTPException(status_code=403, detail="You do not have access to this invoice")

pdf = FPDF()
pdf.add_page()
pdf.set_auto_page_break(auto=True, margin=15)

# Header
pdf.set_font("Helvetica", "B", 20)
pdf.cell(0, 12, "RouteShare", ln=True, align="C")
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 6, "AI-Based Empty-Return Truck Sharing Platform", ln=True, align="C")
pdf.ln(5)
pdf.line(10, pdf.get_y(), 200, pdf.get_y())
pdf.ln(5)

# Invoice title
pdf.set_font("Helvetica", "B", 16)
pdf.cell(0, 10, "FREIGHT INVOICE", ln=True, align="C")
pdf.ln(3)

# Booking info
pdf.set_font("Helvetica", "B", 11)
pdf.cell(0, 8, f"Booking Reference: {booking.booking_ref}", ln=True)
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 7, f>Date: {booking.created_at.strftime('%d %b %Y, %I:%M %p'))", ln=True)
pdf.cell(0, 7, f>Status: {booking.status.value.replace('_', ' ').title()}", ln=True)
pdf.ln(5)

# Route info
pdf.set_font("Helvetica", "B", 12)
pdf.cell(0, 8, "Route Details", ln=True)
pdf.line(10, pdf.get_y(), 200, pdf.get_y())
pdf.ln(2)
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 7, f"Route Code: {route.route_code}", ln=True)
pdf.cell(0, 7, f"Origin: {route.source_city}", ln=True)
pdf.cell(0, 7, f"Destination: {route.destination_city}", ln=True)
if route.intermediate_hubs:
    pdf.cell(0, 7, f"Via: {route.intermediate_hubs}", ln=True)
pdf.cell(0, 7, f"Return Date: {route.return_date.strftime('%d %b %Y')}", ln=True)
pdf.ln(5)

# Shipment info
pdf.set_font("Helvetica", "B", 12)
pdf.cell(0, 8, "Shipment Details", ln=True)
pdf.line(10, pdf.get_y(), 200, pdf.get_y())
pdf.ln(2)
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 7, f"Pickup Location: {booking.pickup_location}", ln=True)
pdf.cell(0, 7, f"Delivery Location: {booking.delivery_location}", ln=True)
pdf.cell(0, 7, f"Cargo Weight: {booking.cargo_weight_kg} kg", ln=True)
if booking.cargo_volume_cbm:
    pdf.cell(0, 7, f"Cargo Volume: {booking.cargo_volume_cbm} CBM", ln=True)
pdf.ln(5)

# Cost breakdown
pdf.set_font("Helvetica", "B", 12)
pdf.cell(0, 8, "Cost Breakdown", ln=True)
pdf.line(10, pdf.get_y(), 200, pdf.get_y())
pdf.ln(2)
pdf.set_font("Helvetica", "", 10)
pdf.cell(95, 7, "Description", border=1, align="C")
pdf.cell(30, 7, "Weight (kg)", border=1, align="C")
pdf.cell(30, 7, "Rate/kg", border=1, align="C")
pdf.cell(35, 7, "Amount", border=1, align="C")
pdf.ln()
pdf.cell(95, 7, f"{booking.pickup_location} to {booking.delivery_location}", border=1)
pdf.cell(30, 7, f"{booking.cargo_weight_kg}", border=1, align="C")
pdf.cell(30, 7, f"Rs.{route.rate_per_kg}", border=1, align="C")
pdf.cell(35, 7, f"Rs.{booking.cost}", border=1, align="C")
pdf.ln()
pdf.set_font("Helvetica", "B", 11)
pdf.cell(155, 8, "Total Amount:", align="R")

```

```

pdf.cell(35, 8, f"Rs.{booking.cost}", border=1, align="C")
pdf.ln(8)

# Parties
pdf.set_font("Helvetica", "B", 12)
pdf.cell(0, 8, "Parties", ln=True)
pdf.line(10, pdf.get_y(), 200, pdf.get_y())
pdf.ln(2)
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 7, f"Shipper: {shipper.full_name} ({shipper.email})", ln=True)
pdf.cell(0, 7, f"Provider: {provider.full_name} ({provider.email})", ln=True)
if provider.company_name:
    pdf.cell(0, 7, f"Company: {provider.company_name}", ln=True)
pdf.ln(5)

# Timestamps
if booking.picked_up_at:
    pdf.cell(0, 7, f"Picked up at: {booking.picked_up_at.strftime('%d %b %Y, %I:%M %p')}",
ln=True)
if booking.delivered_at:
    pdf.cell(0, 7, f"Delivered at: {booking.delivered_at.strftime('%d %b %Y, %I:%M %p')}",
ln=True)
pdf.ln(10)

# Footer
pdf.set_font("Helvetica", "I", 9)
pdf.cell(0, 6, "This is a system-generated invoice from RouteShare.", ln=True, align="C")
pdf.cell(0, 6, "Thank you for using RouteShare!", ln=True, align="C")

pdf_bytes = pdf.output()
buffer = io.BytesIO(pdf_bytes)
buffer.seek(0)

return StreamingResponse(
    buffer,
    media_type="application/pdf",
    headers={"Content-Disposition": f"attachment;
filename=invoice_{booking.booking_ref}.pdf"},
)

```

12. backend/app/routers/vehicles.py

```

from typing import List
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.deps import require_roles
from app.models import User, UserRole, Vehicle, AuditLog
from app.schemas import VehicleCreate, VehicleOut

router = APIRouter(prefix="/api/vehicles", tags=["Vehicles"])

@router.post("", response_model=VehicleOut, status_code=201)
def create_vehicle(payload: VehicleCreate, db: Session = Depends(get_db), current_user: User =
Depends(require_roles(UserRole.truck_provider))):
    existing = db.query(Vehicle).filter(Vehicle.vehicle_number ==
payload.vehicle_number).first()
    if existing:
        raise HTTPException(status_code=400, detail="A vehicle with this number is already
registered")
    vehicle = Vehicle(
        provider_id=current_user.id,
        vehicle_number=payload.vehicle_number.strip().upper(),
        vehicle_type=payload.vehicle_type.strip(),
        capacity_kg=payload.capacity_kg,
    )
    db.add(vehicle)
    db.commit()
    db.refresh(vehicle)
    db.add(AuditLog(user_id=current_user.id, action="VEHICLE_REGISTERED", details=f"Vehicle
{vehicle.vehicle_number}"))
    db.commit()

```

```

    return vehicle

@router.get("/my", response_model=List[VehicleOut])
def my_vehicles(db: Session = Depends(get_db), current_user: User =
Depends(require_roles(UserRole.truck_provider))):
    return db.query(Vehicle).filter(Vehicle.provider_id == current_user.id, Vehicle.is_active
== True).order_by(Vehicle.created_at.desc()).all()

@router.delete("/{vehicle_id}", status_code=204)
def delete_vehicle(vehicle_id: str, db: Session = Depends(get_db), current_user: User =
Depends(require_roles(UserRole.truck_provider))):
    vehicle = db.query(Vehicle).filter(Vehicle.id == vehicle_id, Vehicle.provider_id ==
current_user.id).first()
    if not vehicle:
        raise HTTPException(status_code=404, detail="Vehicle not found")
    vehicle.is_active = False
    db.commit()
    return None

```

13. backend/app/routers/drivers.py

```

from datetime import datetime, timezone
from typing import List

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from app.database import get_db
from app.deps import require_roles
from app.models import User, UserRole, TruckRoute, Booking, RouteStatus, BookingStatus,
AuditLog
from app.schemas import TruckRouteOut, BookingOut

router = APIRouter(prefix="/api/driver", tags=["Driver Management"])

@router.get("/trips", response_model=List[TruckRouteOut])
def assigned_trips(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.driver)),
):
    """FR-23: The system shall display all assigned trips."""
    return (
        db.query(TruckRoute)
        .filter(TruckRoute.driver_id == current_user.id)
        .order_by(TruckRoute.return_date.asc())
        .all()
    )

@router.get("/trips/{route_id}/bookings", response_model=List[BookingOut])
def trip_bookings(
    route_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.driver)),
):
    route = db.query(TruckRoute).filter(TruckRoute.id == route_id, TruckRoute.driver_id ==
current_user.id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Trip not found or not assigned to you")
    return route.bookings

def _assert_one_active_trip(db: Session, driver_id: str, exclude_route_id: str):
    """BR: A driver may be assigned to multiple trips but only one active trip at a time."""
    active = (
        db.query(TruckRoute)
        .join(Booking, Booking.route_id == TruckRoute.id)
        .filter(
            TruckRoute.driver_id == driver_id,
            TruckRoute.id != exclude_route_id,

```

```

        Booking.status == BookingStatus.picked_up,
    )
    .first()
)
if active:
    raise HTTPException(
        status_code=400,
        detail="You already have an active trip in progress. Complete it before starting
another.",
    )

```

```

@router.patch("/bookings/{booking_id}/pickup", response_model=BookingOut)
def confirm_pickup(

```

```

    booking_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.driver)),
):
    """FR-25: The system shall allow drivers to confirm cargo pickup."""
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Booking not found")
    if booking.route.driver_id != current_user.id:
        raise HTTPException(status_code=403, detail="This trip is not assigned to you")
    if booking.status != BookingStatus.confirmed:
        raise HTTPException(status_code=400, detail=f"Cannot pick up a booking with status
{booking.status.value}")

```

```

    _assert_one_active_trip(db, current_user.id, booking.route_id)

```

```

    booking.status = BookingStatus.picked_up
    booking.picked_up_at = datetime.now(timezone.utc)
    db.add(AuditLog(user_id=current_user.id, action="CARGO_PICKED_UP", details=f"Booking
{booking.booking_ref}"))
    db.commit()
    db.refresh(booking)
    return booking

```

```

@router.patch("/bookings/{booking_id}/deliver", response_model=BookingOut)
def confirm_delivery(

```

```

    booking_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.driver)),
):
    """FR-26, FR-27, FR-28: Confirm cargo delivery, notify shipper, record completion time."""
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Booking not found")
    if booking.route.driver_id != current_user.id:
        raise HTTPException(status_code=403, detail="This trip is not assigned to you")
    if booking.status != BookingStatus.picked_up:
        raise HTTPException(status_code=400, detail="Cargo must be picked up before it can be
marked delivered")

```

```

    booking.status = BookingStatus.delivered
    booking.delivered_at = datetime.now(timezone.utc)

```

```

    # FR-27: notify shipper of status change (in-app notification record via audit log;
    # an SMTP/SMS integration point for production deployment).
    db.add(AuditLog(user_id=current_user.id, action="CARGO_DELIVERED", details=f"Booking
{booking.booking_ref}"))
    db.commit()
    db.refresh(booking)
    return booking

```

```

@router.patch("/trips/{route_id}/complete", response_model=TruckRouteOut)
def complete_trip(

```

```

    route_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.driver)),
):

```

```

        """FR-28: Report route completion once all bookings on the trip are delivered."""
        route = db.query(TruckRoute).filter(TruckRoute.id == route_id, TruckRoute.driver_id ==
current_user.id).first()
        if not route:
            raise HTTPException(status_code=404, detail="Trip not found or not assigned to you")

        undelivered = [b for b in route.bookings if b.status not in (BookingStatus.delivered,
BookingStatus.cancelled)]
        if undelivered:
            raise HTTPException(status_code=400, detail="All bookings must be delivered or
cancelled before completing the trip")

        route.status = RouteStatus.completed
        db.add(AuditLog(user_id=current_user.id, action="ROUTE_COMPLETED", details=f"Route
{route.route_code}"))
        db.commit()
        db.refresh(route)
        return route

```

14. backend/app/routers/search.py

```

from datetime import datetime, timezone
from typing import List, Optional

from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from app.database import get_db
from app.deps import require_roles
from app.models import User, UserRole, TruckRoute, RouteStatus
from app.schemas import SearchResult, TruckRouteOut

router = APIRouter(prefix="/api/search", tags=["Freight Search & Matching"])

def route_corridor_matches(route: TruckRoute, pickup: str, destination: str) -> bool:
    """
    AI Route Matching Engine (simulated).

    Compares the shipper's requested pickup/destination against the truck's
    return-route corridor: source city, destination city, and any
    intermediate transit hubs. A match occurs when both the pickup and
    delivery locations fall within the route's corridor, in the correct
    directional order.
    """
    corridor = [route.source_city.strip().lower()]
    if route.intermediate_hubs:
        corridor += [h.strip().lower() for h in route.intermediate_hubs.split(",") if
h.strip()]
    corridor.append(route.destination_city.strip().lower())

    pickup_l = pickup.strip().lower()
    destination_l = destination.strip().lower()

    if pickup_l not in corridor or destination_l not in corridor:
        return False

    # Directional check: pickup must occur at or before destination along the corridor.
    return corridor.index(pickup_l) <= corridor.index(destination_l)

@router.get("", response_model=List[SearchResult])
def search_freight(
    pickup_location: str = Query(..., min_length=2),
    destination_location: str = Query(..., min_length=2),
    cargo_weight_kg: float = Query(..., gt=0),
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.shipper, UserRole.admin)),
):
    """

```

```

FR-12, FR-13, FR-14, FR-15, FR-16: Accept a freight search request, run it
through the AI route-matching engine against active routes, verify
capacity, and return only routes that satisfy both route and capacity
requirements. Returns an empty list (with 200 OK) when nothing matches,
which the frontend surfaces as a "no matching route" notice.
"""
now = datetime.now(timezone.utc)
candidate_routes = (
    db.query(TruckRoute)
    .filter(
        TruckRoute.status == RouteStatus.active,
        TruckRoute.available_capacity_kg >= cargo_weight_kg,
        TruckRoute.return_date >= now,
    )
    .all()
)

results: List[SearchResult] = []
for route in candidate_routes:
    if route_corridor_matches(route, pickup_location, destination_location):
        results.append(
            SearchResult(
                route=TruckRouteOut.model_validate(route),
                provider_name=route.provider.company_name or route.provider.full_name,
                estimated_cost_for_query=round(cargo_weight_kg * route.rate_per_kg, 2),
            )
        )

results.sort(key=lambda r: r.estimated_cost_for_query or 0)
return results

```

15. backend/app/routers/admin.py

```

from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func

from app.database import get_db
from app.deps import require_roles
from app.models import User, UserRole, TruckRoute, Booking, RouteStatus, BookingStatus,
AuditLog
from app.schemas import UserOut, TruckRouteOut, BookingOut, ReportSummary

router = APIRouter(prefix="/api/admin", tags=["Administration"])

@router.get("/users", response_model=List[UserOut])
def list_users(
    role: Optional[UserRole] = None,
    search: Optional[str] = Query(None, description="Search by name or email"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.admin)),
):
    query = db.query(User)
    if role:
        query = query.filter(User.role == role)
    if search:
        like = f"%{search}%"
        query = query.filter((User.full_name.ilike(like)) | (User.email.ilike(like)))
    return query.order_by(User.created_at.desc()).offset((page - 1) *
page_size).limit(page_size).all()

@router.patch("/users/{user_id}/deactivate", response_model=UserOut)
def deactivate_user(
    user_id: str,
    db: Session = Depends(get_db),

```

```

        current_user: User = Depends(require_roles(UserRole.admin)),
    ):
        user = db.query(User).filter(User.id == user_id).first()
        if not user:
            raise HTTPException(status_code=404, detail="User not found")
        user.is_active = False
        db.add(AuditLog(user_id=current_user.id, action="USER_DEACTIVATED", details=f"User {user.email} deactivated"))
        db.commit()
        db.refresh(user)
        return user

@router.delete("/users/{user_id}", status_code=204)
def delete_user(
    user_id: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.admin)),
):
    """BR: Only Administrators shall have permission to delete user accounts."""
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    if user.id == current_user.id:
        raise HTTPException(status_code=400, detail="You cannot delete your own account")

    db.add(AuditLog(user_id=current_user.id, action="USER_DELETED", details=f"User {user.email} deleted"))
    db.delete(user)
    db.commit()
    return None

@router.get("/routes", response_model=List[TruckRouteOut])
def all_routes(
    status_filter: Optional[RouteStatus] = Query(None, alias="status"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.admin)),
):
    query = db.query(TruckRoute)
    if status_filter:
        query = query.filter(TruckRoute.status == status_filter)
    return query.order_by(TruckRoute.created_at.desc()).offset((page - 1) *
page_size).limit(page_size).all()

@router.get("/bookings", response_model=List[BookingOut])
def all_bookings(
    status_filter: Optional[BookingStatus] = Query(None, alias="status"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.admin)),
):
    query = db.query(Booking)
    if status_filter:
        query = query.filter(Booking.status == status_filter)
    return query.order_by(Booking.created_at.desc()).offset((page - 1) *
page_size).limit(page_size).all()

@router.get("/reports/summary", response_model=ReportSummary)
def report_summary(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(UserRole.admin)),
):
    """Route utilization, booking trends, vehicle occupancy, and revenue statistics."""
    total_users = db.query(func.count(User.id)).scalar() or 0
    total_providers = db.query(func.count(User.id)).filter(User.role ==
UserRole.truck_provider).scalar() or 0

```

```

    total_shippers = db.query(func.count(User.id)).filter(User.role ==
UserRole.shipper).scalar() or 0
    total_drivers = db.query(func.count(User.id)).filter(User.role == UserRole.driver).scalar()
or 0

    total_routes = db.query(func.count(TruckRoute.id)).scalar() or 0
    active_routes = db.query(func.count(TruckRoute.id)).filter(TruckRoute.status ==
RouteStatus.active).scalar() or 0
    completed_routes = db.query(func.count(TruckRoute.id)).filter(TruckRoute.status ==
RouteStatus.completed).scalar() or 0

    total_bookings = db.query(func.count(Booking.id)).scalar() or 0
    total_revenue = db.query(func.coalesce(func.sum(Booking.cost), 0.0)).filter(
        Booking.status != BookingStatus.cancelled
    ).scalar() or 0.0

    routes = db.query(TruckRoute).all()
    if routes:
        utilization = sum(
            (r.total_capacity_kg - r.available_capacity_kg) / r.total_capacity_kg
            for r in routes
            if r.total_capacity_kg > 0
        ) / len(routes) * 100
    else:
        utilization = 0.0

    return ReportSummary(
        total_users=total_users,
        total_providers=total_providers,
        total_shippers=total_shippers,
        total_drivers=total_drivers,
        total_routes=total_routes,
        active_routes=active_routes,
        completed_routes=completed_routes,
        total_bookings=total_bookings,
        total_revenue=round(total_revenue, 2),
        average_capacity_utilization_pct=round(utilization, 2),
    )

```

16. backend/seed_data.py

```

"""
Sample test data seeder.

Run with: python seed_data.py
Creates one user of each role plus a couple of active routes so the
system can be manually tested end-to-end immediately after setup.
"""
from datetime import datetime, timedelta, timezone

from app.database import SessionLocal, Base, engine
from app.models import User, UserRole, TruckRoute, RouteStatus
from app.security import hash_password

Base.metadata.create_all(bind=engine)
db = SessionLocal()

SAMPLE_USERS = [
    dict(full_name="Admin User", email="admin@freight.test", phone="+91-9000000001",
        password="Admin@1234", role=UserRole.admin, company_name=None),
    dict(full_name="Kumar Logistics", email="provider@freight.test", phone="+91-9000000002",
        password="Provider@123", role=UserRole.truck_provider, company_name="Kumar Logistics
Pvt Ltd"),
    dict(full_name="Anitha Textiles", email="shipper@freight.test", phone="+91-9000000003",
        password="Shipper@123", role=UserRole.shipper, company_name="Anitha Textiles"),
    dict(full_name="Suresh Driver", email="driver@freight.test", phone="+91-9000000004",
        password="Driver@1234", role=UserRole.driver, company_name=None),
]

created_users = {}
for u in SAMPLE_USERS:

```

```

existing = db.query(User).filter(User.email == u["email"]).first()
if existing:
    created_users[u["role"]] = existing
    continue
user = User(
    full_name=u["full_name"],
    email=u["email"],
    phone=u["phone"],
    password_hash=hash_password(u["password"]),
    role=u["role"],
    company_name=u["company_name"],
)
db.add(user)
db.commit()
db.refresh(user)
created_users[u["role"]] = user
print(f"Created {u['role'].value}: {u['email']} / {u['password']}")

provider = created_users[UserRole.truck_provider]
driver = created_users[UserRole.driver]

if not db.query(TruckRoute).filter(TruckRoute.route_code == "RT-SAMPLE01").first():
    route = TruckRoute(
        route_code="RT-SAMPLE01",
        provider_id=provider.id,
        driver_id=driver.id,
        source_city="Coimbatore",
        destination_city="Chennai",
        intermediate_hubs="Salem, Vellore",
        return_date=datetime.now(timezone.utc) + timedelta(days=3),
        total_capacity_kg=5000,
        available_capacity_kg=5000,
        rate_per_kg=8.5,
        status=RouteStatus.active,
    )
    db.add(route)
    db.commit()
    print("Created sample route RT-SAMPLE01: Coimbatore -> Chennai via Salem, Vellore")

db.close()
print("\nSeed complete. You can now log in with any of the sample accounts above.")

```

17. frontend/src/services/api.js

```

import axios from "axios";

const api = axios.create({
  baseURL: "/api",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("rs_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("rs_token");
      localStorage.removeItem("rs_user");
      if (!window.location.pathname.startsWith("/login")) {
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  }
);

```

```

export function extractErrorMessage(error) {
  const detail = error?.response?.data?.detail;
  if (!detail) return "Something went wrong. Please try again.";
  if (typeof detail === "string") return detail;
  if (Array.isArray(detail)) {
    return detail.map((d) => d.message || d.msg).join(" · ");
  }
  return "Something went wrong. Please try again.";
}

export const authApi = {
  register: (payload) => api.post("/auth/register", payload),
  login: (payload) => api.post("/auth/login", payload),
  me: () => api.get("/auth/me"),
};

export const vehiclesApi = {
  create: (payload) => api.post("/vehicles", payload),
  mine: () => api.get("/vehicles/my"),
  remove: (id) => api.delete(`/vehicles/${id}`),
};

export const routesApi = {
  create: (payload) => api.post("/routes", payload),
  mine: () => api.get("/routes/my"),
  get: (id) => api.get(`/routes/${id}`),
  update: (id, payload) => api.put(`/routes/${id}`, payload),
  remove: (id) => api.delete(`/routes/${id}`),
  assignDriver: (id, driverId) => api.patch(`/routes/${id}/assign-driver`, { driver_id: driverId }),
  availableDrivers: () => api.get("/routes/available-drivers"),
};

export const searchApi = {
  search: (params) => api.get("/search", { params }),
};

export const bookingsApi = {
  create: (payload) => api.post("/bookings", payload),
  mine: () => api.get("/bookings/my"),
  providerBookings: () => api.get("/bookings/provider"),
  cancel: (id) => api.patch(`/bookings/${id}/cancel`),
  downloadInvoice: (id) => api.get(`/bookings/${id}/invoice`, { responseType: "blob" }),
};

export const driverApi = {
  trips: () => api.get("/driver/trips"),
  tripBookings: (routeId) => api.get(`/driver/trips/${routeId}/bookings`),
  confirmPickup: (bookingId) => api.patch(`/driver/bookings/${bookingId}/pickup`),
  confirmDelivery: (bookingId) => api.patch(`/driver/bookings/${bookingId}/deliver`),
  completeTrip: (routeId) => api.patch(`/driver/trips/${routeId}/complete`),
};

export const adminApi = {
  users: (params) => api.get("/admin/users", { params }),
  deactivateUser: (id) => api.patch(`/admin/users/${id}/deactivate`),
  deleteUser: (id) => api.delete(`/admin/users/${id}`),
  routes: (params) => api.get("/admin/routes", { params }),
  bookings: (params) => api.get("/admin/bookings", { params }),
  summary: () => api.get("/admin/reports/summary"),
};

export default api;

```

18. frontend/src/context/AuthContext.jsx

```

import { createContext, useContext, useState, useCallback } from "react";
import { authApi } from "../services/api";

const AuthContext = createContext(null);

```

```

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const stored = localStorage.getItem("rs_user");
    return stored ? JSON.parse(stored) : null;
  });

  const login = useCallback(async (email, password) => {
    const { data } = await authApi.login({ email, password });
    localStorage.setItem("rs_token", data.access_token);
    localStorage.setItem("rs_user", JSON.stringify(data.user));
    setUser(data.user);
    return data.user;
  }, []);

  const register = useCallback(async (payload) => {
    await authApi.register(payload);
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem("rs_token");
    localStorage.removeItem("rs_user");
    setUser(null);
  }, []);

  return (
    <AuthContext.Provider value={{ user, login, register, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}

```

19. frontend/src/context/ToastContext.jsx

```

import { createContext, useContext, useCallback, useState } from "react";

const ToastContext = createContext(null);

export function ToastProvider({ children }) {
  const [toasts, setToasts] = useState([]);

  const pushToast = useCallback((message, variant = "info") => {
    const id = Math.random().toString(36).slice(2);
    setToasts((prev) => [...prev, { id, message, variant }]);
    setTimeout(() => {
      setToasts((prev) => prev.filter((t) => t.id !== id));
    }, 4500);
  }, []);

  return (
    <ToastContext.Provider value={{ pushToast }}>
      {children}
      <div className="fixed bottom-5 right-5 z-50 flex flex-col gap-2 w-80 max-w-[90vw]">
        {toasts.map((t) => (
          <div
            key={t.id}
            role="status"
            className={`rounded-lg px-4 py-3 shadow-lg text-sm font-medium border-l-4 bg-white
              animate-[fadein_0.2s_ease-out] ${
                t.variant === "success"
                  ? "border-emerald-500 text-emerald-900"
                  : t.variant === "error"
                  ? "border-red-500 text-red-900"
                  : "border-navy-600 text-navy-900"
              }`}
          >
            {t.message}
          </div>
        ))}
      </div>
    </ToastContext.Provider>
  );
}

```

```

        {t.message}
      </div>
    )))
  </div>
</ToastContext.Provider>
);
}

export function useToast() {
  const ctx = useContext(ToastContext);
  if (!ctx) throw new Error("useToast must be used within ToastProvider");
  return ctx;
}

```

20. frontend/src/components/ProtectedRoute.jsx

```

import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function ProtectedRoute({ children, allowedRoles }) {
  const { user } = useAuth();

  if (!user) {
    return <Navigate to="/login" replace />;
  }

  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }

  return children;
}

```

21. frontend/src/components/Navbar.jsx

```

import { NavLink, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

const ROLE_HOME = {
  truck_provider: "/provider",
  shipper: "/shipper",
  driver: "/driver",
  admin: "/admin",
};

const ROLE_LABEL = {
  truck_provider: "Truck Provider",
  shipper: "Shipper",
  driver: "Driver",
  admin: "Administrator",
};

export default function Navbar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <header className="bg-navy-950 text-white sticky top-0 z-40 border-b border-navy-700/60">
      <div className="max-w-6xl mx-auto px-5 h-16 flex items-center justify-between">
        <NavLink to={user ? ROLE_HOME[user.role] : "/"} className="flex items-center gap-2 focus-ring rounded">
          <span className="w-8 h-8 rounded-md bg-amber-500 flex items-center justify-center font-display font-bold text-navy-950">
            R

```

```

    </span>
    <span className="font-display font-semibold text-lg tracking-tight">RouteShare</span>
  </NavLink>

  {user && (
    <div className="flex items-center gap-5">
      <div className="hidden sm:flex flex-col items-end leading-tight">
        <span className="text-sm font-medium">{user.full_name}</span>
        <span className="text-xs text-amber-400 font-mono uppercase tracking-wide">
          {ROLE_LABEL[user.role]}
        </span>
      </div>
      <button
        onClick={handleLogout}
        className="focus-ring text-sm font-medium px-3 py-1.5 rounded-md border border-
navy-600 hover:bg-navy-800 transition-colors"
      >
        Log out
      </button>
    </div>
  )}
</div>
</header>
);
}

```

22. frontend/src/components/Ui.jsx

```

export function Card({ children, className = "" }) {
  return (
    <div className={`bg-white rounded-xl border border-navy-900/10 shadow-sm
${className}`}>{children}</div>
  );
}

export function Button({ children, variant = "primary", className = "", ...props }) {
  const variants = {
    primary: "bg-navy-900 text-white hover:bg-navy-800",
    accent: "bg-amber-500 text-navy-950 hover:bg-amber-400 font-semibold",
    outline: "border border-navy-900/20 text-navy-900 hover:bg-navy-900/5",
    danger: "bg-red-600 text-white hover:bg-red-500",
    ghost: "text-navy-700 hover:bg-navy-900/5",
  };
  return (
    <button
      className={`focus-ring inline-flex items-center justify-center gap-2 rounded-lg px-4 py-2
text-sm font-medium transition-colors disabled:opacity-50 disabled:cursor-not-allowed
${variants[variant]} ${className}`}
      {...props}
    >
      {children}
    </button>
  );
}

export function Badge({ children, tone = "default" }) {
  const tones = {
    default: "bg-navy-900/10 text-navy-800",
    active: "bg-emerald-100 text-emerald-800",
    completed: "bg-blue-100 text-blue-800",
    cancelled: "bg-red-100 text-red-700",
    pending: "bg-amber-100 text-amber-800",
    confirmed: "bg-blue-100 text-blue-800",
    picked_up: "bg-purple-100 text-purple-800",
    delivered: "bg-emerald-100 text-emerald-800",
  };
  return (
    <span className={`inline-block px-2.5 py-1 rounded-full text-xs font-semibold capitalize
${tones[children] || tones[tone]}`}
      >{String(children).replace("_", " ")}
    </span>
  );
}

```

```

export function Input({ label, error, className = "", ...props }) {
  return (
    <label className="block">
      {label} && <span className="block text-sm font-medium text-navy-800 mb-1">{label}</span>
      <input
        className={`focus-ring w-full rounded-lg border px-3 py-2 text-sm bg-white ${
          error ? "border-red-400" : "border-navy-900/15"
        } ${className}`}
        {...props}
      />
      {error} && <span className="text-xs text-red-600 mt-1 block">{error}</span>
    </label>
  );
}

export function Select({ label, error, children, className = "", ...props }) {
  return (
    <label className="block">
      {label} && <span className="block text-sm font-medium text-navy-800 mb-1">{label}</span>
      <select
        className={`focus-ring w-full rounded-lg border px-3 py-2 text-sm bg-white ${
          error ? "border-red-400" : "border-navy-900/15"
        } ${className}`}
        {...props}
      >
        {children}
      </select>
      {error} && <span className="text-xs text-red-600 mt-1 block">{error}</span>
    </label>
  );
}

export function Spinner({ className = "" }) {
  return (
    <div
      className={`inline-block w-5 h-5 border-2 border-navy-900/20 border-t-navy-900 rounded-
full animate-spin ${className}`}
      role="status"
      aria-label="Loading"
    />
  );
}

export function EmptyState({ title, description, action }) {
  return (
    <div className="text-center py-14 px-6">
      <div className="w-12 h-12 mx-auto mb-4 rounded-full bg-navy-900/5 flex items-center
justify-center text-navy-400 font-display text-xl">
        —
      </div>
      <h3 className="font-display font-semibold text-navy-900 mb-1">{title}</h3>
      {description} && <p className="text-sm text-navy-600 mb-4 max-w-sm mx-
auto">{description}</p>
      {action}
    </div>
  );
}

export function Modal({ open, onClose, title, children }) {
  if (!open) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
      <div className="absolute inset-0 bg-navy-950/50" onClick={onClose} />
      <div className="relative bg-white rounded-xl shadow-xl w-full max-w-md p-6">
        <h3 className="font-display font-semibold text-lg text-navy-900 mb-3">{title}</h3>
        {children}
      </div>
    </div>
  );
}

```

23. frontend/src/App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import { ToastProvider } from "../context/ToastContext";
import Navbar from "../components/Navbar";
import ProtectedRoute from "../components/ProtectedRoute";

import Landing from "../pages/Landing";
import Login from "../pages/Login";
import Register from "../pages/Register";
import ProviderDashboard from "../pages/ProviderDashboard";
import ShipperDashboard from "../pages/ShipperDashboard";
import DriverDashboard from "../pages/DriverDashboard";
import AdminDashboard from "../pages/AdminDashboard";
import NotFound from "../pages/NotFound";

export default function App() {
  return (
    <AuthProvider>
      <ToastProvider>
        <BrowserRouter>
          <Navbar />
          <Routes>
            <Route path="/" element={<Landing />} />
            <Route path="/login" element={<Login />} />
            <Route path="/register" element={<Register />} />
            <Route
              path="/provider"
              element={
                <ProtectedRoute allowedRoles={["truck_provider"]}>
                  <ProviderDashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path="/shipper"
              element={
                <ProtectedRoute allowedRoles={["shipper"]}>
                  <ShipperDashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path="/driver"
              element={
                <ProtectedRoute allowedRoles={["driver"]}>
                  <DriverDashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path="/admin"
              element={
                <ProtectedRoute allowedRoles={["admin"]}>
                  <AdminDashboard />
                </ProtectedRoute>
              }
            />
            <Route path="*" element={<NotFound />} />
          </Routes>
        </BrowserRouter>
      </ToastProvider>
    </AuthProvider>
  );
}
```

24. frontend/src/pages/Landing.jsx

```
import { Link } from "react-router-dom";
import { Button } from "../components/UI";
```

```

export default function Landing() {
  return (
    <div className="bg-navy-950 text-white min-h-[calc(100vh-4rem)]">
      <div className="max-w-5xl mx-auto px-5 py-24 text-center">
        <span className="font-mono text-xs text-amber-400 tracking-widest uppercase">Empty-
return freight, matched</span>
        <h1 className="font-display font-semibold text-4xl sm:text-5xl mt-4 leading-tight">
          Turn every return trip
          <br />
          into a paying route.
        </h1>
        <p className="text-navy-300 max-w-xl mx-auto mt-5">
          RouteShare connects truck providers with unused return-journey capacity to shippers
who need
          affordable freight space – matched automatically by corridor and capacity.
        </p>
        <div className="flex items-center justify-center gap-3 mt-8">
          <Link to="/register">
            <Button variant="accent">Get started</Button>
          </Link>
          <Link to="/login">
            <Button variant="outline" className="border-white/20 text-white hover:bg-white/10">
              Log in
            </Button>
          </Link>
        </div>

        <div className="grid sm:grid-cols-3 gap-5 mt-20 text-left">
          <FeatureCard title="Register a route" text="Truck providers publish source, hubs,
destination, and available capacity." />
          <FeatureCard title="Get matched" text="Shippers search by corridor and cargo weight –
the engine surfaces only routes that fit." />
          <FeatureCard title="Book with confidence" text="Capacity updates in real time, so
overbooking is never possible." />
        </div>
      </div>
    </div>
  );
}

function FeatureCard({ title, text }) {
  return (
    <div className="border border-white/10 rounded-xl p-5 bg-white/[0.03]">
      <h3 className="font-display font-semibold mb-2">{title}</h3>
      <p className="text-sm text-navy-300">{text}</p>
    </div>
  );
}

```

25. frontend/src/pages/Login.jsx

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";
import { Card, Button, Input, Spinner } from "../components/UI";
import { extractErrorMessage } from "../services/api";

const ROLE_HOME = {
  truck_provider: "/provider",
  shipper: "/shipper",
  driver: "/driver",
  admin: "/admin",
};

export default function Login() {
  const { login } = useAuth();
  const { pushToast } = useToast();
  const navigate = useNavigate();
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);

```

```

const validate = () => {
  const e = {};
  if (!/^\\S+@\\S+\\.\\S+$/.test(form.email)) e.email = "Enter a valid email address";
  if (!form.password) e.password = "Password is required";
  setErrors(e);
  return Object.keys(e).length === 0;
};

const handleSubmit = async (evt) => {
  evt.preventDefault();
  if (!validate()) return;
  setLoading(true);
  try {
    const user = await login(form.email, form.password);
    pushToast(`Welcome back, ${user.full_name.split(" ")[0]}`, "success");
    navigate(ROLE_HOME[user.role] || "/");
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
  } finally {
    setLoading(false);
  }
};

return (
  <div className="min-h-[calc(100vh-4rem)] flex items-center justify-center px-4 bg-slate-50">
    <Card className="w-full max-w-sm p-7">
      <h1 className="font-display font-semibold text-2xl text-navy-950 mb-1">Log in</h1>
      <p className="text-sm text-navy-600 mb-6">Access your RouteShare dashboard.</p>

      <form onSubmit={handleSubmit} className="space-y-4" noValidate>
        <Input
          label="Email address"
          type="email"
          value={form.email}
          onChange={(e) => setForm({ ...form, email: e.target.value })}
          error={errors.email}
          autoComplete="email"
        />
        <Input
          label="Password"
          type="password"
          value={form.password}
          onChange={(e) => setForm({ ...form, password: e.target.value })}
          error={errors.password}
          autoComplete="current-password"
        />
        <Button type="submit" variant="accent" className="w-full" disabled={loading}>
          {loading ? <Spinner /> : "Log in"}
        </Button>
      </form>

      <p className="text-sm text-navy-600 mt-6 text-center">
        New to RouteShare?{" "}
        <Link to="/register" className="text-navy-950 font-medium underline underline-offset-2">
          Create an account
        </Link>
      </p>
    </Card>
  </div>
);
}

```

26. frontend/src/pages/Register.jsx

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
import { useToast } from "../../context/ToastContext";
import { Card, Button, Input, Select, Spinner } from "../../components/UI";

```

```

import { extractErrorMessage } from "../services/api";

export default function Register() {
  const { register } = useAuth();
  const { pushToast } = useToast();
  const navigate = useNavigate();
  const [form, setForm] = useState({
    full_name: "",
    email: "",
    phone: "",
    password: "",
    role: "shipper",
    company_name: "",
  });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);

  const set = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const validate = () => {
    const e = {};
    if (form.full_name.trim().length < 2) e.full_name = "Enter your full name";
    if (!/^\\S+@\\S+\\.\\S+$/.test(form.email)) e.email = "Enter a valid email address";
    if (!/^\\+?[0-9\\-\\s]{7,20}$/.test(form.phone)) e.phone = "Enter a valid phone number";
    if (form.password.length < 8) e.password = "At least 8 characters";
    else if (!/[A-Z]/.test(form.password) || ![a-z]/.test(form.password) ||
    !/\\d/.test(form.password)) {
      e.password = "Use upper, lower case letters and a number";
    }
    setErrors(e);
    return Object.keys(e).length === 0;
  };

  const handleSubmit = async (evt) => {
    evt.preventDefault();
    if (!validate()) return;
    setLoading(true);
    try {
      await register({ ...form, company_name: form.company_name || null });
      pushToast("Account created. You can log in now.", "success");
      navigate("/login");
    } catch (err) {
      pushToast(extractErrorMessage(err), "error");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-[calc(100vh-4rem)] flex items-center justify-center px-4 py-10 bg-slate-50">
      <Card className="w-full max-w-md p-7">
        <h1 className="font-display font-semibold text-2xl text-navy-950 mb-1">Create your
account</h1>
        <p className="text-sm text-navy-600 mb-6">Join as a Truck Provider, Shipper, or
Driver.</p>

        <form onSubmit={handleSubmit} className="space-y-4" noValidate>
          <Select label="I am a" value={form.role} onChange={set("role")}>
            <option value="shipper">Shipper – I need freight transportation</option>
            <option value="truck_provider">Truck Provider – I have return-route
capacity</option>
            <option value="driver">Driver – I operate a registered truck</option>
          </Select>

          <Input label="Full name" value={form.full_name} onChange={set("full_name")}
error={errors.full_name} />
          <Input label="Email address" type="email" value={form.email} onChange={set("email")}
error={errors.email} autoComplete="email" />
          <Input label="Phone number" value={form.phone} onChange={set("phone")}
error={errors.phone} placeholder="+91 90000 00000" />

          {form.role === "truck_provider" && (

```

```

        <Input
          label="Company name (optional)"
          value={form.company_name}
          onChange={set("company_name")}
          placeholder="e.g. Kumar Logistics Pvt Ltd"
        />
      )}

      <Input
        label="Password"
        type="password"
        value={form.password}
        onChange={set("password")}
        error={errors.password}
        autoComplete="new-password"
      />

      <Button type="submit" variant="accent" className="w-full" disabled={loading}>
        {loading ? <Spinner /> : "Create account"}
      </Button>
    </form>

    <p className="text-sm text-navy-600 mt-6 text-center">
      Already registered?{" "}
      <Link to="/login" className="text-navy-950 font-medium underline underline-offset-2">
        Log in
      </Link>
    </p>
  </Card>
</div>
);
}

```

27. frontend/src/pages/ShipperDashboard.jsx

```

import { useEffect, useState } from "react";
import { searchApi, bookingsApi, extractErrorMessage } from "../services/api";
import { useToast } from "../context/ToastContext";
import { Card, Button, Input, Badge, Spinner, EmptyState, Modal } from "../components/UI";

export default function ShipperDashboard() {
  const { pushToast } = useToast();
  const [searchForm, setSearchForm] = useState({ pickup_location: "", destination_location: "",
  cargo_weight_kg: "" });
  const [results, setResults] = useState(null);
  const [searching, setSearching] = useState(false);
  const [bookingTarget, setBookingTarget] = useState(null);
  const [bookingForm, setBookingForm] = useState({ pickup_location: "", delivery_location: "",
  cargo_volume_cbm: "" });
  const [submitting, setSubmitting] = useState(false);
  const [history, setHistory] = useState([]);
  const [loadingHistory, setLoadingHistory] = useState(true);

  const loadHistory = async () => {
    setLoadingHistory(true);
    try {
      const { data } = await bookingsApi.mine();
      setHistory(data);
    } catch (err) {
      pushToast(extractErrorMessage(err), "error");
    } finally {
      setLoadingHistory(false);
    }
  };

  const downloadInvoice = async (bookingId, bookingRef) => {
    try {
      const response = await bookingsApi.downloadInvoice(bookingId);
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = document.createElement('a');
      link.href = url;
      link.setAttribute('download', `invoice_${bookingRef}.pdf`);
    }
  };
}

```

```

        document.body.appendChild(link);
        link.click();
        link.parentNode.removeChild(link);
        window.URL.revokeObjectURL(url);
    } catch (err) {
        pushToast(extractErrorMessage(err), 'error');
    }
};

useEffect(() => {
    loadHistory();
}, []);

const handleSearch = async (evt) => {
    evt.preventDefault();
    setSearching(true);
    setResults(null);
    try {
        const { data } = await searchApi.search({
            pickup_location: searchForm.pickup_location,
            destination_location: searchForm.destination_location,
            cargo_weight_kg: Number(searchForm.cargo_weight_kg),
        });
        setResults(data);
        if (data.length === 0) {
            pushToast("No matching return routes found for this search.", "info");
        }
    } catch (err) {
        pushToast(extractErrorMessage(err), "error");
    } finally {
        setSearching(false);
    }
};

const openBooking = (result) => {
    setBookingTarget(result);
    setBookingForm({
        pickup_location: searchForm.pickup_location,
        delivery_location: searchForm.destination_location,
        cargo_volume_cbm: "",
    });
};

const confirmBooking = async () => {
    setSubmitting(true);
    try {
        const { data } = await bookingsApi.create({
            route_id: bookingTarget.route.id,
            pickup_location: bookingForm.pickup_location,
            delivery_location: bookingForm.delivery_location,
            cargo_weight_kg: Number(searchForm.cargo_weight_kg),
            cargo_volume_cbm: bookingForm.cargo_volume_cbm ? Number(bookingForm.cargo_volume_cbm) :
null,
        });
        pushToast(`Booking confirmed - reference ${data.booking_ref}`, "success");
        setBookingTarget(null);
        handleSearch({ preventDefault: () => {} });
        loadHistory();
    } catch (err) {
        pushToast(extractErrorMessage(err), "error");
    } finally {
        setSubmitting(false);
    }
};

const cancelBooking = async (id, ref) => {
    try {
        await bookingsApi.cancel(id);
        pushToast(`Booking ${ref} cancelled.`, "success");
        loadHistory();
    } catch (err) {
        pushToast(extractErrorMessage(err), "error");
    }
};

```

```

return (
  <div className="max-w-6xl mx-auto px-5 py-8 space-y-8">
    <div>
      <h1 className="font-display font-semibold text-2xl text-navy-950">Shipper
Dashboard</h1>
      <p className="text-navy-600 text-sm mt-1">Search empty-return capacity along your
freight corridor and reserve space.</p>
    </div>

    <Card className="p-6">
      <h2 className="font-display font-semibold text-navy-900 mb-4">Search freight
opportunities</h2>
      <form onSubmit={handleSearch} className="grid sm:grid-cols-4 gap-3 items-end">
        <Input
          label="Pickup city"
          required
          value={searchForm.pickup_location}
          onChange={(e) => setSearchForm({ ...searchForm, pickup_location: e.target.value })}
        />
        <Input
          label="Delivery city"
          required
          value={searchForm.destination_location}
          onChange={(e) => setSearchForm({ ...searchForm, destination_location:
e.target.value })}
        />
        <Input
          label="Cargo weight (kg)"
          type="number"
          min="1"
          required
          value={searchForm.cargo_weight_kg}
          onChange={(e) => setSearchForm({ ...searchForm, cargo_weight_kg: e.target.value })}
        />
        <Button type="submit" variant="accent" disabled={searching}>
          {searching ? <Spinner /> : "Search"}
        </Button>
      </form>
    </Card>

    {results !== null && (
      <div className="space-y-3">
        <h2 className="font-display font-semibold text-navy-900">Matching routes
({results.length})</h2>
        {results.length === 0 ? (
          <Card>
            <EmptyState
              title="No matching route available"
              description="Try a nearby city, a different pickup/delivery pair, or check back
later as new return routes are registered."
            />
          </Card>
        ) : (
          results.map((r) => (
            <Card key={r.route.id} className="p-5 flex items-center justify-between gap-4
flex-wrap">
              <div>
                <div className="flex items-center gap-2 mb-1">
                  <span className="font-mono text-xs text-navy-
500">{r.route.route_code}</span>
                  <Badge tone="active">provider: {r.provider_name}</Badge>
                </div>
                <h3 className="font-semibold text-navy-950">
                  {r.route.source_city} → {r.route.destination_city}
                </h3>
                <p className="text-sm text-navy-600">
                  {r.route.available_capacity_kg} kg available · returns {new
Date(r.route.return_date).toLocaleDateString()}
                </p>
              </div>
              <div className="text-right">
                <div className="font-display font-semibold text-lg text-navy-
950">₹{r.estimated_cost_for_query}</div>
                <Button variant="accent" className="mt-2" onClick={() => openBooking(r)}>

```

```

        Reserve space
      </Button>
    </div>
  </Card>
)
))
}}
</div>
)}

<div className="space-y-3">
  <h2 className="font-display font-semibold text-navy-900">Booking history</h2>
  {loadingHistory ? (
    <Spinner />
  ) : history.length === 0 ? (
    <Card>
      <EmptyState title="No bookings yet" description="Your reserved freight bookings
will appear here." />
    </Card>
  ) : (
    <Card className="divide-y divide-navy-900/10">
      {history.map((b) => (
        <div key={b.id} className="p-4 flex items-center justify-between gap-3 flex-
wrap">
          <div>
            <span className="font-mono text-xs text-navy-500">{b.booking_ref}</span>
            <p className="text-sm text-navy-800">
              {b.pickup_location} → {b.delivery_location} · {b.cargo_weight_kg} kg ·
₹{b.cost}
            </p>
          </div>
          <div className="flex items-center gap-3">
            <Badge tone={b.status}>{b.status}</Badge>
            {[ 'confirmed', 'picked_up', 'delivered' ].includes(b.status) && (
              <Button variant="outline" className="text-xs px-3 py-1.5" onClick={() =>
downloadInvoice(b.id, b.booking_ref)}>
                Invoice PDF
              </Button>
            )}
            {(b.status === "confirmed" || b.status === "pending") && (
              <Button variant="outline" className="text-xs px-3 py-1.5" onClick={() =>
cancelBooking(b.id, b.booking_ref)}>
                Cancel
              </Button>
            )}
          </div>
        </div>
      )})}
    </Card>
  )}
</div>

<Modal open={!!bookingTarget} onClose={() => setBookingTarget(null)} title="Confirm your
booking">
  {bookingTarget && (
    <div className="space-y-4">
      <p className="text-sm text-navy-600">
        Reserving <strong>{searchForm.cargo_weight_kg} kg</strong> on route{" "}
        <strong>{bookingTarget.route.route_code}</strong> for an estimated{" "}
        <strong>₹{bookingTarget.estimated_cost_for_query}</strong>.
      </p>
      <Input
        label="Pickup location"
        value={bookingForm.pickup_location}
        onChange={(e) => setBookingForm({ ...bookingForm, pickup_location: e.target.value
)}}
      />
      <Input
        label="Delivery location"
        value={bookingForm.delivery_location}
        onChange={(e) => setBookingForm({ ...bookingForm, delivery_location:
e.target.value })}
      />
      <Input
        label="Cargo volume in cbm (optional)"
        type="number"

```

```

        min="0.1"
        step="0.1"
        value={bookingForm.cargo_volume_cbm}
        onChange={(e) => setBookingForm({...bookingForm, cargo_volume_cbm:
e.target.value })}
      />
      <div className="flex gap-3 justify-end">
        <Button variant="ghost" onClick={() => setBookingTarget(null)}>
          Cancel
        </Button>
        <Button variant="accent" onClick={confirmBooking} disabled={submitting}>
          {submitting ? <Spinner /> : "Confirm booking"}
        </Button>
      </div>
    </div>
  )}
</Modal>
</div>
);
}

```

28. frontend/src/pages/ProviderDashboard.jsx

```

import { useEffect, useState } from "react";
import { routesApi, bookingsApi, vehiclesApi } from "../services/api";
import { extractErrorMessage } from "../services/api";
import { useToast } from "../context/ToastContext";
import { Card, Button, Input, Select, Badge, Spinner, EmptyState, Modal } from
"../components/UI";

const emptyForm = {
  vehicle_id: "",
  source_city: "",
  destination_city: "",
  intermediate_hubs: "",
  return_date: "",
  total_capacity_kg: "",
  rate_per_kg: "",
};

const emptyVehicleForm = {
  vehicle_number: "",
  vehicle_type: "Truck",
  capacity_kg: "",
};

export default function ProviderDashboard() {
  const { pushToast } = useToast();
  const [routes, setRoutes] = useState([]);
  const [bookings, setBookings] = useState([]);
  const [vehicles, setVehicles] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState(emptyForm);
  const [vehicleForm, setVehicleForm] = useState(emptyVehicleForm);
  const [submitting, setSubmitting] = useState(false);
  const [vehicleSubmitting, setVehicleSubmitting] = useState(false);
  const [confirmDelete, setConfirmDelete] = useState(null);

  const [assignModalOpen, setAssignModalOpen] = useState(false);
  const [assignTargetRoute, setAssignTargetRoute] = useState(null);
  const [availableDrivers, setAvailableDrivers] = useState([]);
  const [selectedDriver, setSelectedDriver] = useState("");
  const [assigningDriver, setAssigningDriver] = useState(false);

  const loadData = async () => {
    setLoading(true);
    try {
      const [r, b, v] = await Promise.all([routesApi.mine(), bookingsApi.providerBookings(),
vehiclesApi.mine()]);
      setRoutes(r.data);
      setBookings(b.data);
      setVehicles(v.data);
    }
  }
}

```

```

    } catch (err) {
      pushToast(extractErrorMessage(err), "error");
    } finally {
      setLoading(false);
    }
  };

useEffect(() => {
  loadData();
}, []);

const handleCreateRoute = async (evt) => {
  evt.preventDefault();
  setSubmitting(true);
  try {
    await routesApi.create({
      ...form,
      total_capacity_kg: Number(form.total_capacity_kg),
      rate_per_kg: Number(form.rate_per_kg),
      return_date: new Date(form.return_date).toISOString(),
      vehicle_id: form.vehicle_id ? Number(form.vehicle_id) : undefined,
    });
    pushToast("Return route registered successfully.", "success");
    setForm(emptyForm);
    loadData();
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
  } finally {
    setSubmitting(false);
  }
};

const handleDeleteRoute = async () => {
  try {
    await routesApi.remove(confirmDelete.id);
    pushToast(`Route ${confirmDelete.route_code} deleted.`, "success");
    setConfirmDelete(null);
    loadData();
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
    setConfirmDelete(null);
  }
};

const handleCreateVehicle = async (evt) => {
  evt.preventDefault();
  setVehicleSubmitting(true);
  try {
    await vehiclesApi.create({
      ...vehicleForm,
      capacity_kg: Number(vehicleForm.capacity_kg),
    });
    pushToast("Vehicle registered successfully.", "success");
    setVehicleForm(emptyVehicleForm);
    loadData();
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
  } finally {
    setVehicleSubmitting(false);
  }
};

const handleDeleteVehicle = async (id) => {
  try {
    await vehiclesApi.remove(id);
    pushToast("Vehicle removed successfully.", "success");
    loadData();
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
  }
};

const openAssignModal = async (route) => {
  setAssignTargetRoute(route);
};

```

```

    setAssignModalOpen(true);
    try {
      const { data } = await routesApi.availableDrivers();
      setAvailableDrivers(data);
    } catch (err) {
      pushToast(extractErrorMessage(err), "error");
    }
  };

const handleAssignDriver = async () => {
  if (!selectedDriver) return;
  setAssigningDriver(true);
  try {
    await routesApi.assignDriver(assignTargetRoute.id, selectedDriver);
    pushToast("Driver assigned successfully.", "success");
    setAssignModalOpen(false);
    setSelectedDriver("");
    loadData();
  } catch (err) {
    pushToast(extractErrorMessage(err), "error");
  } finally {
    setAssigningDriver(false);
  }
};

const bookingsForRoute = (routeId) => bookings.filter((b) => b.route_id === routeId);

return (
  <div className="max-w-6xl mx-auto px-5 py-8 space-y-8">
    <div>
      <h1 className="font-display font-semibold text-2xl text-navy-950">Truck Provider
Dashboard</h1>
      <p className="text-navy-600 text-sm mt-1">Manage your vehicles, register empty-return
routes, and track bookings.</p>
    </div>

    <Card className="p-6">
      <details open className="group">
        <summary className="font-display font-semibold text-navy-900 cursor-pointer outline-
none list-none flex justify-between items-center">
          <span>My Vehicles</span>
          <span className="text-navy-500 group-open:rotate-180 transition-transform">▼</span>
        </summary>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mt-4">
          <div>
            <h3 className="font-semibold text-sm mb-3">Add a vehicle</h3>
            <form onSubmit={handleCreateVehicle} className="space-y-3">
              <Input label="Vehicle Number" required value={vehicleForm.vehicle_number}
onChange={e => setVehicleForm({ ...vehicleForm, vehicle_number: e.target.value })} />
              <Select label="Vehicle Type" required value={vehicleForm.vehicle_type}
onChange={e => setVehicleForm({ ...vehicleForm, vehicle_type: e.target.value })}>
                <option value="Truck">Truck</option>
                <option value="Mini Truck">Mini Truck</option>
                <option value="Trailer">Trailer</option>
                <option value="Container">Container</option>
              </Select>
              <Input label="Capacity (kg)" type="number" min="1" required
value={vehicleForm.capacity_kg} onChange={e => setVehicleForm({ ...vehicleForm, capacity_kg:
e.target.value })} />
              <Button type="submit" variant="accent" className="w-full"
disabled={vehicleSubmitting}>
                {vehicleSubmitting ? <Spinner /> : "Add Vehicle"}
              </Button>
            </form>
          </div>
          <div>
            <h3 className="font-semibold text-sm mb-3">Registered Vehicles</h3>
            {vehicles.length === 0 ? (
              <p className="text-sm text-navy-500">No vehicles registered yet.</p>
            ) : (
              <div className="space-y-3 max-h-64 overflow-y-auto">
                {vehicles.map(v => (
                  <div key={v.id} className="flex items-center justify-between p-3 bg-navy-
900/5 rounded-lg border border-navy-900/10 text-sm">

```

```

        <div>
            <div className="font-semibold">{v.vehicle_number}</div>
            <div className="text-navy-600">{v.vehicle_type} • {v.capacity_kg} kg
capacity</div>
        </div>
        <Button variant="danger" className="text-xs px-2 py-1" onClick={() =>
handleDeleteVehicle(v.id)}>Delete</Button>
    </div>
    )}}
</div>
)}
</div>
</div>
</details>
</Card>

<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
    <Card className="p-6 lg:col-span-1 h-fit">
        <h2 className="font-display font-semibold text-navy-900 mb-4">Register a return
route</h2>
        <form onSubmit={handleCreateRoute} className="space-y-3">
            <Select label="Vehicle (Optional)" value={form.vehicle_id} onChange={(e) =>
setForm({ ...form, vehicle_id: e.target.value })}>
                <option value="">-- Select a Vehicle --</option>
                {vehicles.map(v => (
                    <option key={v.id} value={v.id}>{v.vehicle_number} ({v.capacity_kg}
kg)</option>
                ))}
            </Select>
            <Input label="Source city" required value={form.source_city} onChange={(e) =>
setForm({ ...form, source_city: e.target.value })} />
            <Input label="Destination city" required value={form.destination_city}
onChange={(e) => setForm({ ...form, destination_city: e.target.value })} />
            <Input
                label="Intermediate hubs (comma-separated)"
                placeholder="e.g. Salem, Vellore"
                value={form.intermediate_hubs}
                onChange={(e) => setForm({ ...form, intermediate_hubs: e.target.value })}
            />
            <Input
                label="Return date"
                type="datetime-local"
                required
                value={form.return_date}
                onChange={(e) => setForm({ ...form, return_date: e.target.value })}
            />
            <Input
                label="Available capacity (kg)"
                type="number"
                min="1"
                required
                value={form.total_capacity_kg}
                onChange={(e) => setForm({ ...form, total_capacity_kg: e.target.value })}
            />
            <Input
                label="Rate per kg (₹)"
                type="number"
                min="0.1"
                step="0.1"
                required
                value={form.rate_per_kg}
                onChange={(e) => setForm({ ...form, rate_per_kg: e.target.value })}
            />
            <Button type="submit" variant="accent" className="w-full" disabled={submitting}>
                {submitting ? <Spinner /> : "Register route"}
            </Button>
        </form>
    </Card>

    <div className="lg:col-span-2 space-y-4">
        <h2 className="font-display font-semibold text-navy-900">Your active routes</h2>
        {loading ? (
            <Spinner />
        ) : routes.length === 0 ? (
            <Card>

```

```

        <EmptyState title="No routes yet" description="Register your first return route
using the form on the left." />
    </Card>
    ) : (
        routes.map((route) => (
            <Card key={route.id} className="p-5">
                <div className="flex items-start justify-between gap-3">
                    <div>
                        <div className="flex items-center gap-2 mb-1">
                            <span className="font-mono text-xs text-navy-500">{route.route_code}</span>
                            <Badge tone={route.status}>{route.status}</Badge>
                        </div>
                        <h3 className="font-semibold text-navy-950">
                            {route.source_city} → {route.destination_city}
                        </h3>
                        {route.intermediate_hubs && (
                            <p className="text-xs text-navy-500">via {route.intermediate_hubs}</p>
                        )}
                        <p className="text-sm text-navy-600 mt-1">
                            Returns {new Date(route.return_date).toLocaleString()}
                        </p>
                        {route.vehicle && (
                            <p className="text-sm font-medium mt-1">Vehicle:
{route.vehicle.vehicle_number}</p>
                        )}
                        {route.driver ? (
                            <p className="text-sm font-medium text-navy-700 mt-1">Driver:
{route.driver.name}</p>
                        ) : (
                            <div className="mt-2">
                                <Button variant="outline" className="text-xs px-3 py-1.5" onClick={() => openAssignModal(route)}>Assign Driver</Button>
                            </div>
                        )}
                    </div>
                    <div>
                        <Button
                            variant="danger"
                            className="text-xs px-3 py-1.5"
                            onClick={() => setConfirmDelete(route)}
                            disabled={route.available_capacity_kg < route.total_capacity_kg}
                            title={
                                route.available_capacity_kg < route.total_capacity_kg
                                ? "Cannot delete a route with active bookings"
                                : "Delete route"
                            }
                        >
                            Delete
                        </Button>
                    </div>
                </div>

                <div className="grid grid-cols-3 gap-3 mt-4 text-sm">
                    <div className="bg-navy-900/5 rounded-lg p-3">
                        <div className="text-navy-500 text-xs">Capacity</div>
                        <div className="font-semibold text-navy-950">
                            {route.available_capacity_kg} / {route.total_capacity_kg} kg
                        </div>
                    </div>
                    <div className="bg-navy-900/5 rounded-lg p-3">
                        <div className="text-navy-500 text-xs">Rate</div>
                        <div className="font-semibold text-navy-950">₹{route.rate_per_kg}/kg</div>
                    </div>
                    <div className="bg-navy-900/5 rounded-lg p-3">
                        <div className="text-navy-500 text-xs">Bookings</div>
                        <div className="font-semibold text-navy-950">{bookingsForRoute(route.id).length}</div>
                    </div>
                </div>

                {bookingsForRoute(route.id).length > 0 && (
                    <div className="mt-4 border-t border-navy-900/10 pt-3 space-y-2">
                        {bookingsForRoute(route.id).map((b) => (
                            <div key={b.id} className="flex items-center justify-between text-sm">
                                <span className="font-mono text-xs text-navy-500">{b.booking_ref}</span>

```

```

                <span className="text-navy-700">{b.cargo_weight_kg} kg</span>
                <Badge tone={b.status}>{b.status}</Badge>
            </div>
        )}
    </div>
    )}
</Card>
))
)}
</div>
</div>

    <Modal open={!!confirmDelete} onClose={() => setConfirmDelete(null)} title="Delete this
route?">
        <p className="text-sm text-navy-600 mb-5">
            This will permanently remove route {confirmDelete?.route_code}. This action cannot be
            undone.
        </p>
        <div className="flex gap-3 justify-end">
            <Button variant="ghost" onClick={() => setConfirmDelete(null)}>
                Cancel
            </Button>
            <Button variant="danger" onClick={handleDeleteRoute}>
                Delete route
            </Button>
        </div>
    </Modal>

    <Modal open={assignModalOpen} onClose={() => setAssignModalOpen(false)} title="Assign
Driver">
        <div className="space-y-4">
            <p className="text-sm text-navy-600">Assign an available driver to route
{assignTargetRoute?.route_code}</p>
            <Select value={selectedDriver} onChange={(e) => setSelectedDriver(e.target.value)}>
                <option value="">-- Select Driver --</option>
                {availableDrivers.map(d => (
                    <option key={d.id} value={d.id}>{d.name} ({d.email})</option>
                ))}
            </Select>
            <div className="flex gap-3 justify-end">
                <Button variant="ghost" onClick={() => setAssignModalOpen(false)}>Cancel</Button>
                <Button variant="accent" onClick={handleAssignDriver} disabled={!selectedDriver ||
assigningDriver}>
                    {assigningDriver ? <Spinner /> : "Assign Driver"}
                </Button>
            </div>
        </div>
    </Modal>
</div>
);
}

```

29. frontend/src/pages/DriverDashboard.jsx

```

import { useEffect, useState } from "react";
import { driverApi, bookingsApi, extractErrorMessage } from "../services/api";
import { useToast } from "../context/ToastContext";
import { Card, Button, Badge, Spinner, EmptyState } from "../components/UI";

export default function DriverDashboard() {
    const { pushToast } = useToast();
    const [trips, setTrips] = useState([]);
    const [tripBookings, setTripBookings] = useState({});
    const [loading, setLoading] = useState(true);
    const [busyId, setBusyId] = useState(null);

    const loadTrips = async () => {
        setLoading(true);
        try {
            const { data } = await driverApi.trips();
            setTrips(data);
            const bookingsByRoute = {};
            await Promise.all(

```

```

        data.map(async (trip) => {
            const { data: bks } = await driverApi.tripBookings(trip.id);
            bookingsByRoute[trip.id] = bks;
        })
    );
    setTripBookings(bookingsByRoute);
} catch (err) {
    pushToast(extractErrorMessage(err), "error");
} finally {
    setLoading(false);
}
};

useEffect(() => {
    loadTrips();
}, []);

const act = async (fn, id, successMsg) => {
    setBusyId(id);
    try {
        await fn();
        pushToast(successMsg, "success");
        loadTrips();
    } catch (err) {
        pushToast(extractErrorMessage(err), "error");
    } finally {
        setBusyId(null);
    }
};

const completeTrip = (routeId, code) =>
    act(() => driverApi.completeTrip(routeId), routeId, `Trip ${code} marked complete.`);

const downloadInvoice = async (bookingId, bookingRef) => {
    try {
        const response = await bookingsApi.downloadInvoice(bookingId);
        const url = window.URL.createObjectURL(new Blob([response.data]));
        const link = document.createElement('a');
        link.href = url;
        link.setAttribute('download', `invoice_${bookingRef}.pdf`);
        document.body.appendChild(link);
        link.click();
        link.parentNode.removeChild(link);
        window.URL.revokeObjectURL(url);
    } catch (err) {
        pushToast(extractErrorMessage(err), 'error');
    }
};

return (
    <div className="max-w-5xl mx-auto px-5 py-8 space-y-6">
        <div>
            <h1 className="font-display font-semibold text-2xl text-navy-950">Driver Dashboard</h1>
            <p className="text-navy-600 text-sm mt-1">View your assigned return trips and update
cargo status.</p>
        </div>

        {loading ? (
            <Spinner />
        ) : trips.length === 0 ? (
            <Card>
                <EmptyState title="No trips assigned" description="Your assigned return trips will
appear here once a provider assigns you to a route." />
            </Card>
        ) : (
            trips.map((trip) => {
                const bookings = tripBookings[trip.id] || [];
                const allSettled = bookings.every((b) => ["delivered",
"cancelled"].includes(b.status));
                return (
                    <Card key={trip.id} className="p-5">
                        <div className="flex items-start justify-between gap-3 flex-wrap">
                            <div>
                                <div className="flex items-center gap-2 mb-1">

```

```

        <span className="font-mono text-xs text-navy-500">{trip.route_code}</span>
        <Badge tone={trip.status}>{trip.status}</Badge>
      </div>
      <h3 className="font-semibold text-navy-950">
        {trip.source_city} → {trip.destination_city}
      </h3>
      <p className="text-sm text-navy-600">Returns {new
Date(trip.return_date).toLocaleString()}</p>
    </div>
    {trip.status === "active" && (
      <Button
        variant="outline"
        className="text-xs"
        disabled={!allSettled || busyId === trip.id}
        onClick={() => completeTrip(trip.id, trip.route_code)}
        title={!allSettled ? "All bookings must be delivered or cancelled first" :
"Mark trip complete"}
      >
        {busyId === trip.id ? <Spinner /> : "Mark trip complete"}
      </Button>
    )}
  </div>

  <div className="mt-4 space-y-2">
    {bookings.length === 0 ? (
      <p className="text-sm text-navy-500">No cargo bookings on this trip yet.</p>
    ) : (
      bookings.map((b) => (
        <div key={b.id} className="flex items-center justify-between gap-3 flex-
wrap bg-navy-900/5 rounded-lg p-3">
          <div className="text-sm">
            <span className="font-mono text-xs text-navy-
500">{b.booking_ref}</span>
            <p className="text-navy-800">
              {b.pickup_location} → {b.delivery_location} · {b.cargo_weight_kg} kg
            </p>
          </div>
          <div className="flex items-center gap-2">
            <Badge tone={b.status}>{b.status}</Badge>
            <Button variant="outline" className="text-xs px-3 py-1.5" onClick={()
=> downloadInvoice(b.id, b.booking_ref)}>
              Invoice PDF
            </Button>
            {b.status === "confirmed" && (
              <Button
                variant="accent"
                className="text-xs px-3 py-1.5"
                disabled={busyId === b.id}
                onClick={() => act(() => driverApi.confirmPickup(b.id), b.id,
"Cargo pickup confirmed.")}>
                >
                {busyId === b.id ? <Spinner /> : "Confirm pickup"}
              </Button>
            )}
            {b.status === "picked_up" && (
              <Button
                variant="accent"
                className="text-xs px-3 py-1.5"
                disabled={busyId === b.id}
                onClick={() => act(() => driverApi.confirmDelivery(b.id), b.id,
"Cargo delivery confirmed.")}>
                >
                {busyId === b.id ? <Spinner /> : "Confirm delivery"}
              </Button>
            )}
          </div>
        </div>
      ))
    )}
  </div>
</Card>
);
})
})
</div>
</div>

```

```
);  
}
```

30. frontend/src/pages/AdminDashboard.jsx

```
import { useEffect, useState } from "react";  
import { adminApi, extractErrorMessage } from "../services/api";  
import { useToast } from "../context/ToastContext";  
import { Card, Button, Badge, Spinner, EmptyState, Modal } from "../components/UI";  
  
const TABS = ["Overview", "Users", "Routes", "Bookings"];  
  
export default function AdminDashboard() {  
  const { pushToast } = useToast();  
  const [tab, setTab] = useState("Overview");  
  const [summary, setSummary] = useState(null);  
  const [users, setUsers] = useState([]);  
  const [routes, setRoutes] = useState([]);  
  const [bookings, setBookings] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [confirmDeleteUser, setConfirmDeleteUser] = useState(null);  
  
  const loadAll = async () => {  
    setLoading(true);  
    try {  
      const [s, u, r, b] = await Promise.all([  
        adminApi.summary(),  
        adminApi.users({}),  
        adminApi.routes({}),  
        adminApi.bookings({}),  
      ]);  
      setSummary(s.data);  
      setUsers(u.data);  
      setRoutes(r.data);  
      setBookings(b.data);  
    } catch (err) {  
      pushToast(extractErrorMessage(err), "error");  
    } finally {  
      setLoading(false);  
    }  
  };  
  
  useEffect(() => {  
    loadAll();  
  }, []);  
  
  const deactivate = async (user) => {  
    try {  
      await adminApi.deactivateUser(user.id);  
      pushToast(`${user.full_name} deactivated.`, "success");  
      loadAll();  
    } catch (err) {  
      pushToast(extractErrorMessage(err), "error");  
    }  
  };  
  
  const deleteUser = async () => {  
    try {  
      await adminApi.deleteUser(confirmDeleteUser.id);  
      pushToast(`${confirmDeleteUser.full_name} deleted.`, "success");  
      setConfirmDeleteUser(null);  
      loadAll();  
    } catch (err) {  
      pushToast(extractErrorMessage(err), "error");  
      setConfirmDeleteUser(null);  
    }  
  };  
  
  return (  
    <div className="max-w-6xl mx-auto px-5 py-8 space-y-6">  
      <div>
```

```

        <h1 className="font-display font-semibold text-2xl text-navy-950">Administrator
Dashboard</h1>
        <p className="text-navy-600 text-sm mt-1">Monitor platform activity, manage users, and
review operational reports.</p>
    </div>

    <div className="flex gap-2 border-b border-navy-900/10">
        {TABS.map((t) => (
            <button
                key={t}
                onClick={() => setTab(t)}
                className={`focus-ring px-4 py-2 text-sm font-medium border-b-2 -mb-px transition-
colors ${
                    tab === t ? "border-amber-500 text-navy-950" : "border-transparent text-navy-500
hover:text-navy-800"
                }`}
            >
                {t}
            </button>
        ))}
    </div>

    {loading ? (
        <Spinner />
    ) : (
        <>
            {tab === "Overview" && summary && (
                <div className="grid sm:grid-cols-2 lg:grid-cols-4 gap-4">
                    <StatCard label="Total users" value={summary.total_users} />
                    <StatCard label="Truck providers" value={summary.total_providers} />
                    <StatCard label="Shippers" value={summary.total_shippers} />
                    <StatCard label="Drivers" value={summary.total_drivers} />
                    <StatCard label="Active routes" value={summary.active_routes} />
                    <StatCard label="Completed routes" value={summary.completed_routes} />
                    <StatCard label="Total bookings" value={summary.total_bookings} />
                    <StatCard label="Revenue" value={`₹${summary.total_revenue.toLocaleString()}`} />
                    <StatCard label="Avg. capacity utilization"
value={` ${summary.average_capacity_utilization_pct}%`} className="lg:col-span-2" />
                </div>
            )}

            {tab === "Users" && (
                <Card className="divide-y divide-navy-900/10">
                    {users.length === 0 ? (
                        <EmptyState title="No users found" />
                    ) : (
                        users.map((u) => (
                            <div key={u.id} className="p-4 flex items-center justify-between gap-3 flex-
wrap">
                                <div>
                                    <p className="font-medium text-navy-950">{u.full_name}</p>
                                    <p className="text-sm text-navy-600">{u.email}</p>
                                </div>
                                <div className="flex items-center gap-2">
                                    <Badge tone="default">{u.role.replace("_", " ")}</Badge>
                                    {!u.is_active && <Badge tone="cancelled">deactivated</Badge>}
                                    {u.is_active && (
                                        <Button variant="outline" className="text-xs px-3 py-1.5" onClick={()
=> deactivate(u)}>
                                            Deactivate
                                        </Button>
                                    )}
                                    <Button variant="danger" className="text-xs px-3 py-1.5" onClick={() =>
setConfirmDeleteUser(u)}>
                                            Delete
                                        </Button>
                                    </div>
                                </div>
                            </div>
                        ))
                    )}
                </Card>
            )}

            {tab === "Routes" && (

```

```

<Card className="divide-y divide-navy-900/10">
  {routes.length === 0 ? (
    <EmptyState title="No routes registered" />
  ) : (
    routes.map((r) => (
      <div key={r.id} className="p-4 flex items-center justify-between gap-3 flex-
wrap">
        <div>
          <span className="font-mono text-xs text-navy-500">{r.route_code}</span>
          <p className="text-navy-800 text-sm">
            {r.source_city} → {r.destination_city} ·
            {r.available_capacity_kg}/{r.total_capacity_kg} kg
          </p>
        </div>
        <Badge tone={r.status}>{r.status}</Badge>
      </div>
    ))
  )}
</Card>
)}

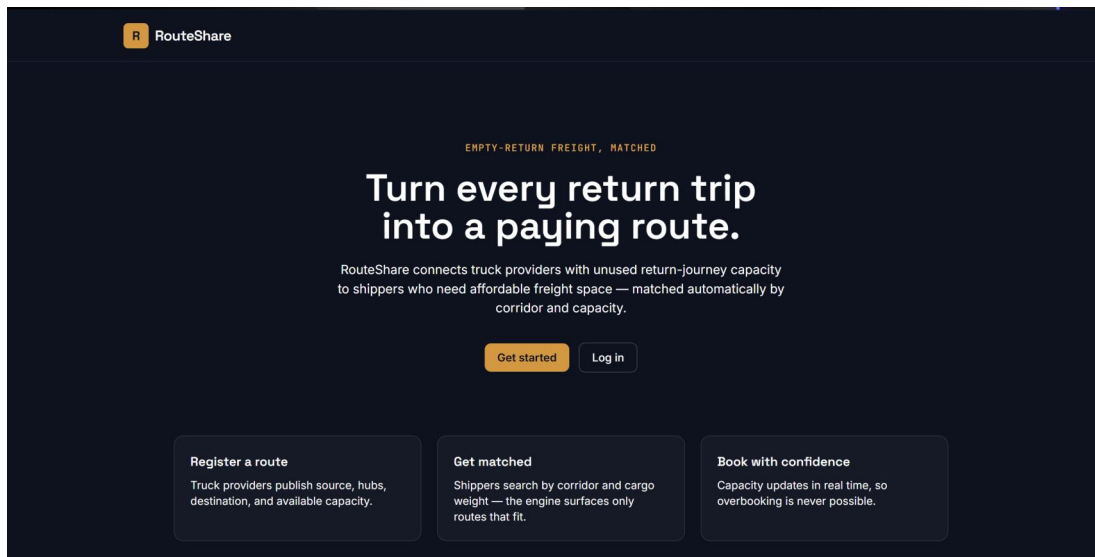
{tab === "Bookings" && (
  <Card className="divide-y divide-navy-900/10">
    {bookings.length === 0 ? (
      <EmptyState title="No bookings yet" />
    ) : (
      bookings.map((b) => (
        <div key={b.id} className="p-4 flex items-center justify-between gap-3 flex-
wrap">
          <div>
            <span className="font-mono text-xs text-navy-500">{b.booking_ref}</span>
            <p className="text-navy-800 text-sm">
              {b.pickup_location} → {b.delivery_location} · {b.cargo_weight_kg} kg ·
              ₹{b.cost}
            </p>
          </div>
          <Badge tone={b.status}>{b.status}</Badge>
        </div>
      ))
    )}
  </Card>
)}
</>
)}

<Modal open={!!confirmDeleteUser} onClose={() => setConfirmDeleteUser(null)}
title="Delete this user?">
  <p className="text-sm text-navy-600 mb-5">
    This will permanently remove {confirmDeleteUser?.full_name}'s account and all related
    data. This action cannot be undone.
  </p>
  <div className="flex gap-3 justify-end">
    <Button variant="ghost" onClick={() => setConfirmDeleteUser(null)}>
      Cancel
    </Button>
    <Button variant="danger" onClick={deleteUser}>
      Delete user
    </Button>
  </div>
</Modal>
</div>
);
}

function StatCard({ label, value, className = "" }) {
  return (
    <Card className={`p-5 ${className}`}>
      <div className="text-navy-500 text-xs font-medium uppercase tracking-wide">{label}</div>
      <div className="font-display font-semibold text-2xl text-navy-950 mt-1">{value}</div>
    </Card>
  );
}

```

Screenshots:



The screenshot shows the "Create your account" form on the RouteShare website. The form is titled "Create your account" and includes the subtext "Join as a Truck Provider, Shipper, or Driver." Below this, there is a dropdown menu for "I am a" with the selected option "Shipper — I need freight transportation". The form contains several input fields: "Full name" (filled with "ROHITHRAGHAV"), "Email address" (filled with "m.s.rohithraghav@gmail.com"), "Phone number" (filled with "9443426267"), and "Password" (filled with "*****"). A "Create account" button is located at the bottom of the form. Below the button, there is a link that says "Already registered? [Log in](#)".

Log in

Access your RouteShare dashboard.

Email address

ROHITHRAGHAV

Password

.....

Log in

New to RouteShare? [Create an account](#)

Create your account

Join as a Truck Provider, Shipper, or Driver.

I am a

Driver — I operate a commercial truck

Full name

Anitha Raj

Email address

anitha@example.com

Phone number

+91 98765 43210

Driver Credentials & Vehicle Details

Driving License (DL) No.

e.g. DL-1420T10012345

Experience (Years)

e.g. 5

Vehicle Number (Optional)

e.g. TN-3B-KL-1001

Vehicle Type

10-Wheeler Truck

Base City / Terminal

e.g. Coimbatore

Emergency Contact Phone

+91 98765 43210

Affiliated Logistics Provider (Optional)

Independent / Self-Employed

Password

.....

Create account

Already registered? [Log in](#)

Shipper Dashboard

Search empty-return capacity along your freight corridor and reserve space.

Search freight opportunities

Pickup city

Coimbatore

Delivery city

Chennai

Cargo weight (kg)

1000

Search

Matching routes (1)

RT-SAPLE01 Provider: Kumar Logistics Pvt Ltd Driver: Suresh Driver TN-3B-KL-1001

₹8500

Coimbatore — Chennai

2900 kg available · returns 10/3/2026

Reserve space

Booking history

BK-3QSV803U Confirmed Route: RT-GBMGZLB0

Kochi — Coimbatore

Weight: 800 kg · 3.2 CBM · Total Cost: ₹6000

Invoice PDF

Cancel

Driver assignment in progress by logistics provider

BK-8GBQWVCU Confirmed Route: RT-4BSZLBAR

RouteShare

Suresh Driver
DRIVER

Log out

S

Suresh Driver

Commercial Driver

On Trip

DL: TN-38-2016-0045678 · Vehicle: TN-38-KL-1001 (10-Wheeler Truck) · Base: Coimbatore

Affiliated Provider

Kumar Logistics Pvt Ltd

Experience

7 Years

Active Trips

2

Completed Trips

0

Total Cargo Consignments

3

Direct Shipper Links

1

My Trips & Manifest

Driver Profile & Vehicle Inputs

Linked Provider & Shippers

Assigned Return Trips & Cargo Manifest

Refresh Trips

RT-SAMPLE1

Active

TN-38-KL-1001 (10-Wheeler Truck)

Provider: Kumar Logistics Pvt Ltd

Mark Trip Complete

Coimbatore → Chennai

Transit via: Salem, Vellore

Scheduled Departure: 10/3/2026, 6:22:42 PM

CARGO CONSIGNMENTS (2)

SHIPPER & CONTACT DETAILS

BK-84CB51XT

Confirmed

1200 kg · 6 CBM

Pickup: Salem → Drop: Chennai

Shipper: Anitha Textiles (Anitha Textiles) · +91-9000000003 · shipper@routeshare.com

Invoice PDF

Confirm Pickup

BK-JZJMF5RP

Picked Up

900 kg · 4.5 CBM

Pickup: Coimbatore → Drop: Salem

Shipper: Anitha Textiles (Anitha Textiles) · +91-9000000003 · shipper@routeshare.com

Invoice PDF

Confirm Delivery

RouteShare

Kumar Logistics
TRUCK PROVIDER

Log out

Truck Provider Dashboard

Manage your vehicles, register empty-return routes, and track bookings.

Fleet Management (2 Vehicles · 1 Drivers)

Add a Fleet Vehicle

Vehicle Number

e.g. TN-38-KL-1001

Vehicle Type

10-Wheeler Truck

Capacity (kg)

e.g. 8000

Add Vehicle

Registered Fleet (2)

TN-33-MX-4521

Mini Truck · 3,500 kg payload

Delete

TN-38-KL-1001

10-Wheeler Truck · 8,000 kg payload

Delete

Fleet & Commercial Drivers (1)

Suresh Driver

On Trip

+91-9000000004 · DL: TN-38-2016-0045678

Truck: TN-38-KL-1001 (10-Wheeler Truck) · 7 yrs exp

Register a return route

Vehicle (Optional)

-- Select a Vehicle --

Source city

Destination city

Intermediate hubs (comma-separated)

e.g. Salem, Vellore

Return date

mm/dd/yyyy, --:--

Available capacity (kg)

Rate per kg (₹)

Register route

Your active routes

RT-6BHGZLB0U

Active

Delete

Kochi → Coimbatore

via Palakkad

Returns 10/7/2026, 6:22:43 PM

Vehicle: TN-38-KL-1001

Assign Driver

Capacity

5200 / 6000 kg

Rate

₹75/kg

Bookings

1

BK-3Q5VB01U

800 kg

Confirmed

RT-4WSZLB4R

Active

Delete

Madurai → Bengaluru

via Dindigul, Salem, Hosur

Returns 10/5/2026, 6:22:43 PM

Vehicle: TN-33-MX-4521

Driver: Suresh Driver

Reassign

+91-9000000004 · DL: TN-38-2016-0045678 · Vehicle: TN-38-KL-1001

Capacity

2000 / 3500 kg

Rate

₹9/kg

Bookings

1

BK-850QWVCU

1500 kg

Confirmed

E-RESOURCE

Github link: <https://github.com/ROHITHRAGHAV33/RouteShare>